THE COMPLETE PROJECT HANDBOOK

CallRoster Pro

An automated call-center staff-scheduling platform, taught from first principles — so you can explain, defend, and discuss every design decision in a senior-level interview.

FastAPI | Google OR-Tools CP-SAT | PostgreSQL | SQLAlchemy 2.0 | Alembic | React 19 | TypeScript | TanStack Query | Vite PWA | JWT Auth | Cloud Run | Supabase | Vercel

Audience: you, the engineer who built it, preparing to explain it to senior engineers.
Promise: read this once and you will be able to draw the architecture from memory, trace any request end-to-end, and justify every technology and trade-off.
Ground rule: every claim here is anchored to real files and functions in this repository. Where something is an assumption, it is labelled as one.

Table of Contents

1 · Executive Overview

1.1 What problem does this project solve?

Imagine a call center — a team of *agents* (the people who answer phones) who must be scheduled across *shifts* (Morning, Evening, Overnight, etc.) seven days a week. Every week a manager must build a **roster**: a grid that says which agent works which shift on which day, while respecting a tangle of competing rules:

- Every agent is legally/contractually owed a **weekly rest day**.
- Agents submit **requests** — "I want Friday off", "I need 3 days of leave", "swap me to the evening shift", "I'll take overtime".
- Certain time slots need a minimum number of agents holding a specific **skill** (e.g. "at least 2 Spanish-speakers between 09:00 and 12:00 on Mondays").
- Some shifts have a hard **headcount cap** (Overnight = 2 people, no more).
- Leave is drawn from a finite annual **balance**.
- When two requests conflict and only one can win, the decision must be **fair** and, all else equal, **first-come-first-served**.

Done by hand on a spreadsheet, this is slow, error-prone, and feels unfair to staff. **CallRoster Pro turns roster-building into a mathematical optimization problem and solves it automatically** — producing a schedule that satisfies every hard rule and honours as many soft requests as possible, then lets the manager review, tweak, publish, and lock it.

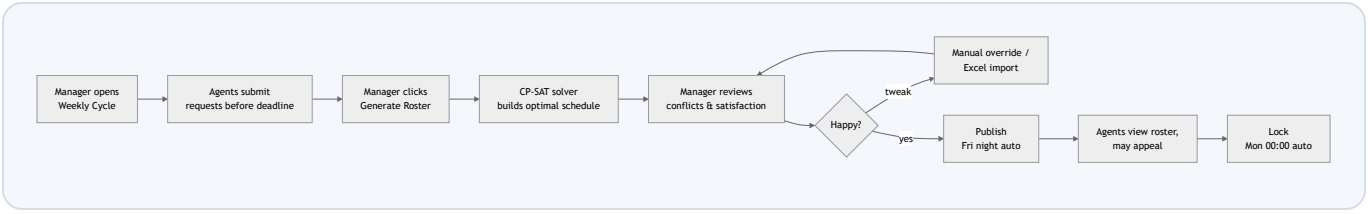
Jargon, immediately unpacked

- **Hard constraint** — a rule that must *never* be broken (a valid schedule with it broken is not a schedule at all). Example: nobody works two shifts on the same day.
- **Soft constraint** — a preference we *want* satisfied but can sacrifice if necessary, each with a "how much does breaking it hurt" number (a *weight*). Example: honouring a leave request.
- **Optimization / solver** — software that searches an astronomically large space of possible schedules and returns the best-scoring one, instead of you trying combinations by hand.

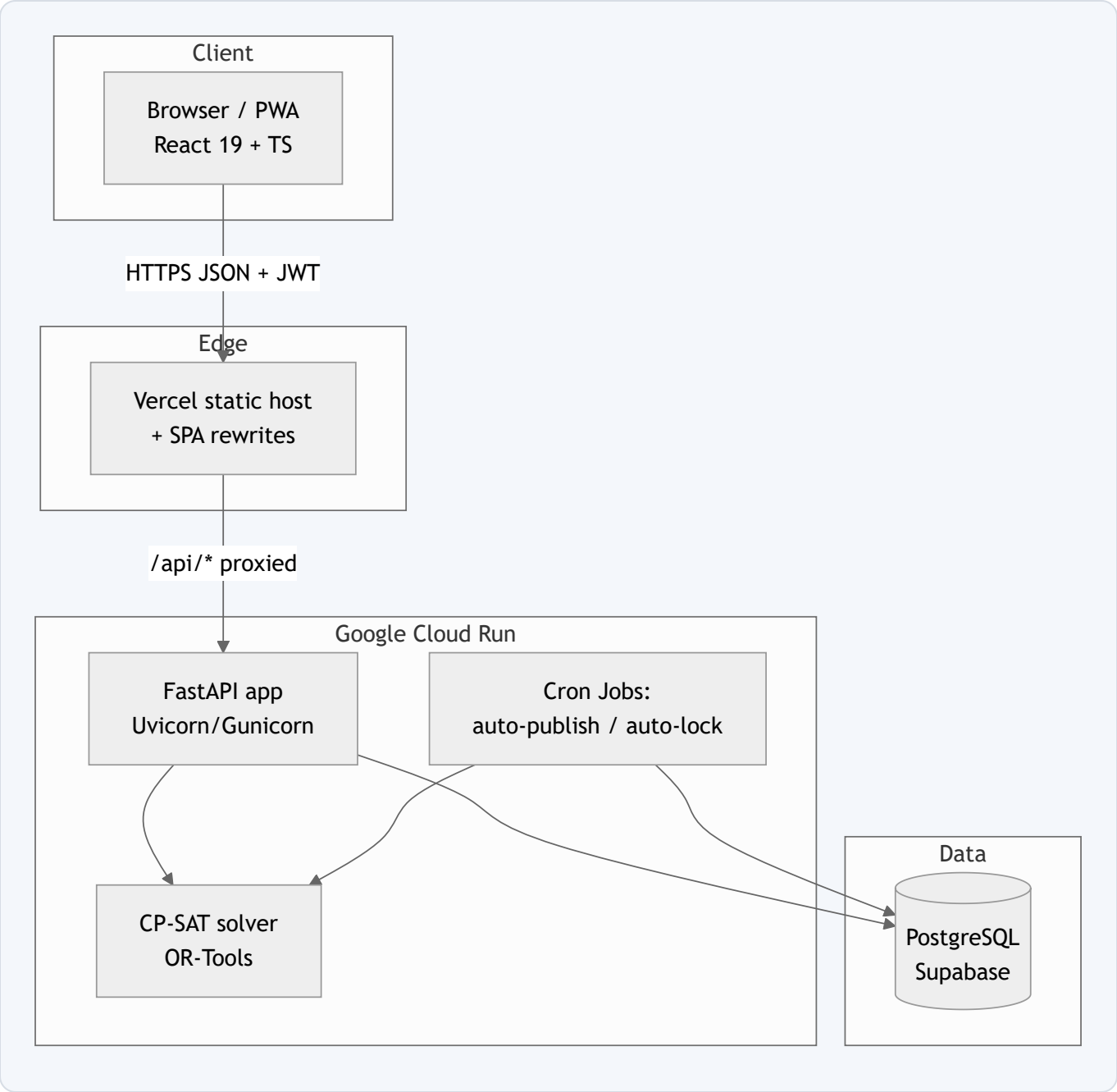
1.2 Who uses it?

Role	What they do in the system	How they authenticate
Manager	Configures skills, shifts, agents, coverage rules, leave balances, solver weights; opens weekly cycles; generates/reviews/overrides/publishes/locks rosters; reviews requests and appeals; reads the audit log.	Logs in (JWT), role = `manager`.
Agent	Submits weekly requests (off-day, leave, shift-change, overtime), views their own roster, leave balance, request outcomes, appeals denied requests, sees their own audit trail.	Logs in (JWT), role = `agent`, linked to an `agent_id`.
Public / anyone	Views the published roster for the current or a given week — no login required.	None (public endpoint).

1.3 The main workflow (weekly cycle)



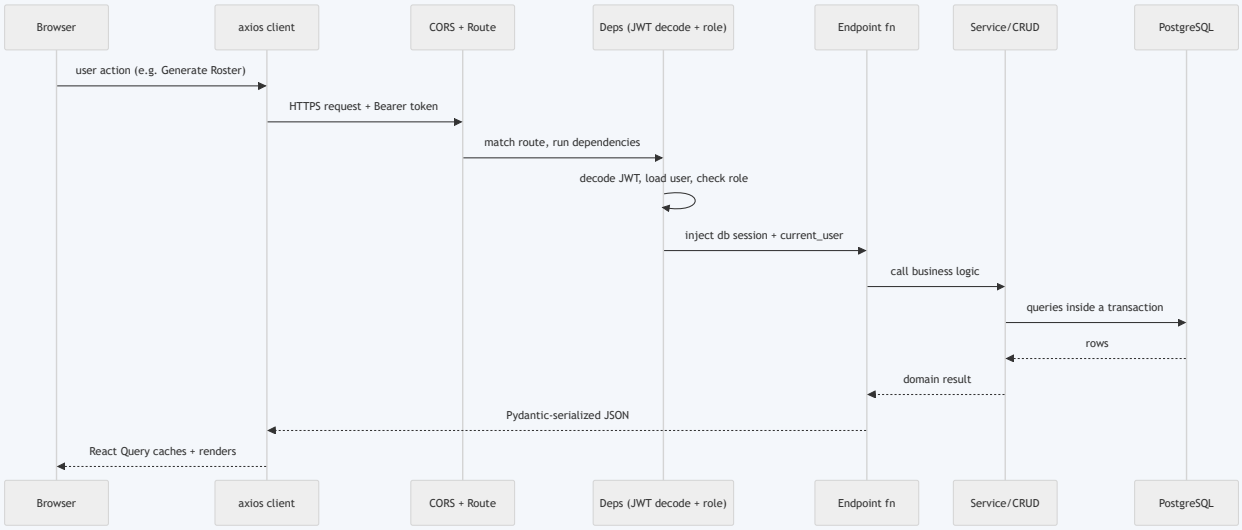
1.4 High-level architecture



1.5 Technologies involved (and the one-line reason for each)

Layer	Technology	Why it's here
Frontend framework	React 19 + TypeScript + Vite	Component UI with compile-time type safety; Vite gives instant dev builds.
Server state	TanStack React Query	Caches API data, dedupes requests, auto-refetches — no manual global store.
Routing / forms	react-router-dom 7, react-hook-form	Declarative routes + performant, uncontrolled form handling.
Styling / PWA	Tailwind CSS 4, vite-plugin-pwa	Utility-first CSS; installable offline-capable app.
API framework	FastAPI (Python)	Async, auto-validated (Pydantic), auto-documented (OpenAPI/Swagger).
The brain	Google OR-Tools CP-SAT	Industrial constraint solver — turns scheduling rules into a solvable model.
ORM	SQLAlchemy 2.0 (typed) + Alembic	Map Python classes to tables; version-control the schema with migrations.
Database	PostgreSQL	Relational integrity, transactions, mature and free.
Auth	JWT (python-jose) + bcrypt (passlib)	Stateless tokens; slow salted password hashing.
Spreadsheets	openpyxl	Managers live in Excel — export/import rosters as `.xlsx`.
Deploy	Cloud Run + Supabase + Vercel	Scale-to-zero backend, managed Postgres, global static frontend — ~$0 for a demo.

1.6 The overall request lifecycle (bird's-eye)



What you should remember

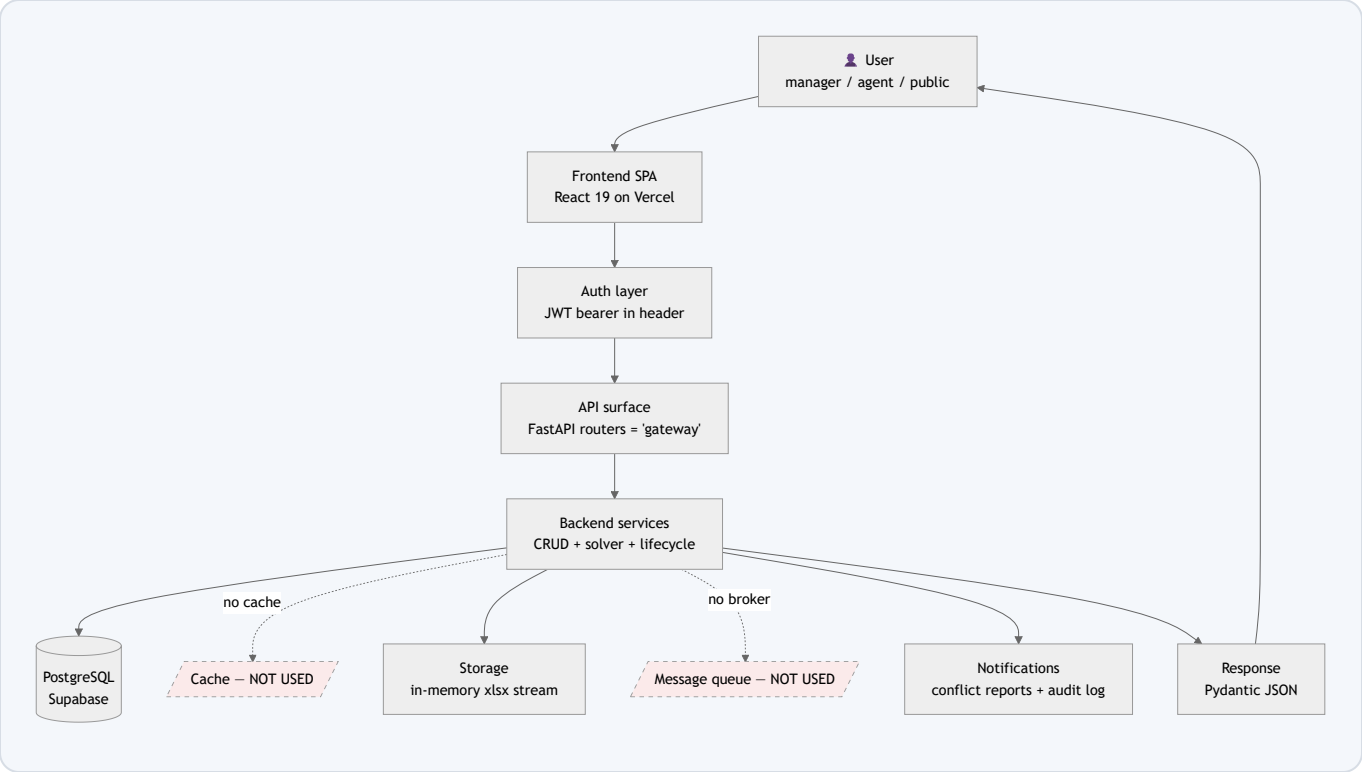
CallRoster Pro is a **constraint-optimization product wearing a CRUD app's clothes**. 90% of the endpoints are ordinary create/read/update/delete. The 10% that matters — and the thing an interviewer will dig into — is `generate_roster` → the CP-SAT model. If you can explain how requests become a weighted objective and rules become constraints, you own this project.

Simple analogy: the solver is a chess engine for schedules. Hard constraints are the rules of chess (illegal moves don't exist). Soft constraints with weights are how it scores a position. It searches, then hands you the best board.

Interviewers actually care about: why a solver instead of hand-written `if/else` logic; how you keep the model feasible; how fairness and first-come-first-served coexist without one silently overriding the other.

2 · Overall Architecture

The generic reference stack in the brief is *User → Frontend → Auth → API Gateway → Backend → Database → Cache → Storage → Queue → Notifications → Response*. This project deliberately implements **some of those layers and consciously omits others**. A senior interviewer respects "we didn't add a cache because we don't need one yet" far more than a cache bolted on for buzzword completeness. So for each layer we state: what it is, what plays that role here (or that it's absent and why), inputs/outputs, and trade-offs.



Layer 1 — User

Responsibility: initiates every action. Three personas (manager, agent, public) with different permissions. **Input:** clicks/forms. **Output:** HTTP requests. **Why it exists:** it's the reason the system exists. There is no "replacement".

Layer 2 — Frontend

What it is: a Single-Page Application (SPA) — one HTML shell that loads a JavaScript bundle which then renders every screen client-side and talks to the backend over JSON. **Here:** React 19 + TypeScript, built by Vite, hosted as static files on Vercel. **Inputs:** user events, cached server data. **Outputs:** HTTPS requests to `/api/*`, rendered DOM. **Why it exists:** separates presentation from data so the backend is a pure JSON API reusable by web, mobile, or the PWA. **Could be replaced by:** server-rendered

templates (Django/Jinja), or a meta-framework (Next.js). **Pros of the SPA choice:** snappy navigation, offline PWA, clean API boundary. **Cons:** initial bundle load, SEO needs care, auth token lives in the browser.

Layer 3 — Authentication

What it is: proving *who* you are and *what* you may do. **Here:** a **JWT** (JSON Web Token) — a signed, self-describing string issued at login and sent on every request in the `Authorization: Bearer <token>` header. FastAPI dependencies decode and verify it. **Inputs:** email+password (login), then the token. **Outputs:** a verified `User` object injected into endpoints, or a 401. **Why stateless tokens:** the server keeps no session table, so any instance can serve any request (horizontal scaling for free). **Could be replaced by:** server-side sessions + cookies, or OAuth/OpenID via an identity provider. **Pros:** stateless, simple, scales. **Cons:** can't instantly revoke a token before it expires; token theft = access until expiry. See §8.

Layer 4 — API Gateway

What a gateway is: a single front door that routes, authenticates, rate-limits, and validates traffic to backend services. **Here there is no separate gateway product** (no Kong/NGINX-as-gateway/AWS API Gateway). FastAPI's router layer plus Cloud Run's built-in HTTPS front end together play that role: routing (`app.include_router` in main.py), validation (Pydantic), CORS (`CORSMiddleware`), TLS termination (Cloud Run). **Why no dedicated gateway:** a single service doesn't need one; adding it would be premature complexity. **When you'd add one:** multiple microservices, centralized rate-limiting, or API-key partners (see §14).

Layer 5 — Backend Services

Responsibility: all business logic. Organized as *routes → services/CRUD → solver/domain → models*. **Inputs:** validated request data + DB session. **Outputs:** persisted state + response schemas. The crown jewel is the **solver service** (`app/solver/`). **Could be replaced by:** a different language/framework, but the layering (thin routes, fat services, pure solver) would carry over. Detailed in §11.

Layer 6 — Database

Here: PostgreSQL, accessed through SQLAlchemy 2.0 ORM, schema-migrated with Alembic. Holds every entity: users, agents, skills, shifts, coverage, cycles, requests, rosters, assignments, conflicts, metrics, appeals, audit log, leave balances, solver weights. **Why relational:** the domain is intensely relational (agents ↔ skills ↔ shifts ↔ coverage) and needs transactional integrity when a roster + its assignments + balance changes must all commit together. Detailed in §7.

Layer 7 — Cache Not used

What a cache is: a fast in-memory store (e.g. Redis) that holds recently-computed answers so you don't recompute or re-query them. **Why absent:** read volume is tiny (one small team), the heaviest computation — the solve — is triggered on demand and stored as rows, and there is no hot read path that repeats an expensive query. Adding Redis now would add an operational dependency for no measurable gain. **What would trigger adding it:** the public roster endpoint getting hammered, or solver results needing memoization. **What we'd lose without ever adding it at scale:** the DB becomes the bottleneck sooner (see §14).

Layer 8 — Storage

What it is: durable blob/file storage (S3, GCS). **Here:** there is no object store; Excel workbooks are generated *in memory* (`BytesIO`) and streamed straight to the browser (`export_roster` in roster.py), and uploads are read into memory and parsed, never saved. **Why:** files are ephemeral artefacts, not assets to retain. **Trade-off:** zero storage cost/complexity; but very large exports would hold memory (fine at this scale).

Layer 9 — Message Queue Not used

What a queue is: a broker (RabbitMQ, SQS, Celery+Redis) that lets you hand off slow work to background workers so the web request returns immediately. **Here:** the solve runs *synchronously* inside the request (capped at 30s, see `max_time_in_seconds`). The only "background" work — auto-publish and auto-lock — runs as **scheduled Cloud Run Jobs / Cloud Scheduler**, not a queue. **Why this is acceptable:** a weekly solve for a small team finishes in well under a second; a queue would add latency and infra for no benefit. **When to add one:** multi-minute solves, or fan-out notifications (email/SMS). See §14/§19.

Layer 10 — Notifications

Here: no email/SMS/push yet. The system's "notification" surface is **in-app**: `ConflictReport` rows (why a request was denied), `SatisfactionMetric` rows (how well each agent did), and the `AuditLog` (who changed what). Agents pull these via the API. **Why:** keeps the demo self-contained; outbound channels are the obvious next feature (§19).

Layer 11 — Response

Every endpoint returns a **Pydantic-validated JSON** body (or a streamed `.xlsx`). Pydantic guarantees the shape matches the declared `response_model` , which is what keeps the TypeScript types on the frontend honest.

What you should remember

This architecture is a **modular monolith**: one deployable backend with clean internal layers, one database, a decoupled SPA. The interview-winning line is: "*We implemented the layers the problem needs and consciously left out cache, queue, and object storage, because at this scale they'd be complexity without benefit — and I can tell you the exact signal that would make me add each one.*"

Common confusion: "no queue" ≠ "no background work". Background work exists (auto-publish/lock) — it's just scheduled, not queued.

3 · Folder Structure

Two top-level apps — `backend/` (Python/FastAPI) and `frontend/` (React/TS) — plus deployment docs and `docker-compose.yml`. Below is the real tree, then every folder explained.

```
Roster-agent/
├── backend/
│   ├── alembic/                # DB migrations (versioned schema history)
│   │   └── versions/           # one file per schema change
│   ├── app/
│   │   ├── api/
│   │   │   ├── deps.py         # shared dependencies: auth, role guards
│   │   │   ├── errors.py       # error helpers
│   │   │   └── routes/         # one router file per resource (13 files)
│   │   ├── core/
│   │   │   ├── config.py       # env-driven settings (Pydantic)
│   │   │   └── security.py     # JWT + bcrypt helpers
│   │   ├── crud/               # DB read/write functions per resource
│   │   ├── db/
│   │   │   ├── base.py         # SQLAlchemy Declarative Base + model imports
│   │   │   └── session.py      # engine + session factory + get_db()
│   │   ├── domain/             # pure business rules (no DB): leave, coverage
│   │   ├── models/             # SQLAlchemy ORM tables + enums
│   │   ├── schemas/            # Pydantic request/response shapes
│   │   ├── services/           # orchestration: excel, lifecycle, leave sync
│   │   ├── solver/             # CP-SAT model (pure) + service (DB glue)
│   │   ├── tests/              # pytest suite (16 test files)
│   │   └── main.py             # FastAPI app assembly + CORS + health
│   └── requirements.txt
├── frontend/
│   └── src/
│       ├── api/                # axios client + typed endpoint functions
│       ├── auth/               # AuthContext (login/logout/token state)
│       ├── components/         # reusable UI: Layout, RosterGrid, ui.tsx
│       ├── lib/                # pure helpers (rosterGrid transforms)
│       ├── pages/              # route screens (agent/ manager/ public/)
│       ├── types/              # shared TypeScript interfaces
│       ├── App.tsx             # route table
│       └── main.tsx            # React root + providers
├── docker-compose.yml          # local Postgres + backend + frontend
└── DEPLOYMENT-*.md             # infra runbooks
```

Backend folders

Folder	Why it exists / what belongs / who touches it / if removed
`alembic/versions/`	**Why:** the database schema is code, versioned like source. Each file is one incremental change (add table, add column). **Interacts with:** `models/` (the target shape) and `db/base.py` (metadata). **If removed:** you lose reproducible schema history — new environments can't be built deterministically and prod upgrades become manual/dangerous.
`app/api/routes/`	**Why:** HTTP entry points, one file per resource (auth, agents, roster, …). Thin: parse → call service → return schema. **Interacts with:** `deps.py`, `crud/`, `services/`, `schemas/`. **If removed:** the app has no surface; nothing is reachable.
`app/api/deps.py`	**Why:** cross-cutting dependencies FastAPI injects — DB session, current user, role guards. Centralizes auth so every route doesn't re-implement it. **If removed:** every protected route breaks; no auth enforcement.
`app/core/`	**Why:** foundational config and security primitives with no business logic. `config.py` reads env vars; `security.py` hashes passwords and mints/verifies JWTs. **If removed:** no settings, no auth crypto.
`app/crud/`	**Why:** the *only* place that issues DB queries for a resource — "Create/Read/Update/Delete". Keeps SQL out of routes. **If removed:** data-access logic scatters into routes, becoming untestable and duplicated.
`app/db/`	**Why:** engine, session factory, and the `Base` all models inherit. `session.py`'s `get_db()` yields a session per request and always closes it. **If removed:** no database connectivity at all.
`app/domain/`	**Why:** *pure* business rules with zero DB or framework imports — `leave.py` (which dates a leave covers) and `shift_coverage.py` (does a shift overlap a slot, incl. overnight wrap). Shared by both the solver and Excel re-validation so the rules can't diverge. **If removed:** the two paths would re-implement the rules and drift out of sync — a classic correctness bug.
`app/models/`	**Why:** the ORM definition of every table and all enums. The single source of truth for the data shape. **If removed:** nothing maps to the DB.
`app/schemas/`	**Why:** Pydantic models defining exactly what each endpoint accepts and returns — the validation + serialization boundary. Distinct from ORM models on purpose (never leak DB internals over the wire). **If removed:** no input validation, no typed responses.
`app/services/`	**Why:** multi-step orchestration that spans several models — `roster_excel.py` (export/import/override), `roster_lifecycle.py` (publish/lock, auto-jobs), `leave_balance_sync.py` (delta balance updates). **If removed:** routes would balloon with tangled logic.

`app/solver/`	**Why:** the optimization engine. `model.py` is *pure* (dataclasses in, dataclasses out, no DB) so it unit-tests without a database; `service.py` is the glue that reads the DB, builds the input, runs `solve`, and writes results. **If removed:** the product loses its entire reason to exist.
`app/tests/`	**Why:** 16 pytest files covering auth, CRUD, solver correctness, lifecycle, excel round-trip, appeals. **If removed:** no safety net; refactoring the solver becomes Russian roulette.

Frontend folders

Folder	Why it exists / if removed
`src/api/`	`client.ts` (axios instance + JWT interceptor + 401 handler) and `endpoints.ts` (one typed function per backend route). **If removed:** no server communication; every component would hand-roll fetches.
`src/auth/`	`AuthContext.tsx` — React Context holding token/role/agentId, exposing `login/logout`. **If removed:** no shared auth state; protected routes can't gate.
`src/components/`	Reusable UI: `Layout` (nav shell), `ProtectedRoute` (role gate), `RosterGrid` (the schedule table), `ui.tsx` (buttons/inputs/cards). **If removed:** duplicated markup everywhere.
`src/lib/`	Pure transforms like `rosterGrid.ts` that reshape flat assignment arrays into a grid. **If removed:** presentation logic leaks into components.
`src/pages/`	One component per screen, grouped by persona (`agent/` , `manager/` , `public/` , plus `manager/config/`). **If removed:** nothing to route to.
`src/types/`	TypeScript interfaces mirroring backend schemas — the contract that catches shape mismatches at compile time. **If removed:** lose type safety across the whole client.

What you should remember

The backend is a textbook **layered architecture**: `routes` (HTTP) → `services/crud` (orchestration + data) → `domain/solver` (pure logic) → `models` (persistence), with `schemas` guarding the edge. The golden rule the folders enforce: **the solver and domain layers know nothing about HTTP or the database**, which is exactly why they're unit-testable in isolation.

Simple analogy: `routes` are waiters, `services/crud` are the kitchen, `domain/solver` is the recipe (pure knowledge), `models` are the pantry. Waiters never cook; the recipe never touches the pantry directly.

4 · Every Important File Explained

For brevity we cover the load-bearing files in depth (purpose, responsibilities, key functions, callers/callees, I/O, lifecycle, interview hooks, pitfalls). The dozens of near-identical CRUD/route files follow the same pattern, described once at the end.

4.1 `backend/app/main.py` — application assembly

Purpose: construct the FastAPI application, attach CORS, register all routers, expose a health check. **Key lines:**

```
app = FastAPI(title="CallRoster Pro API", version="0.1.0")
app.add_middleware(CORSMiddleware, allow_origins=settings.cors_allowed_origins_list,
                   allow_credentials=True, allow_methods=["*"], allow_headers=["*"])
app.include_router(auth.router)   # ... + 12 more routers
@app.get("/api/health")
def health() → dict: return {"status": "ok"}
```

Who calls it: the ASGI server (Uvicorn/Gunicorn) imports `app`. **Who it calls:** every router module. **Lifecycle:** runs once at process start. **Interview Q:** "Why is CORS configured from an env-var list, not `*`?" — because `allow_credentials=True` with wildcard origins is rejected by browsers and is a security anti-pattern; the allowed origins are explicit (localhost dev ports + the deployed Vercel domain). **Common mistake:** forgetting to add the production frontend origin, causing "CORS error" that's really a config gap. **Health check** exists so Cloud Run knows the container is alive.

4.2 `backend/app/core/config.py` — settings

Purpose: read configuration from environment/`.env` via `pydantic-settings`, fail fast if a required var (like `database_url` or `jwt_secret_key`) is missing. **Key detail:** `cors_allowed_origins` is a comma-string parsed into a list by a computed property — env vars are strings, so this is the clean way to pass a list. **Interview Q:** "Why Pydantic settings over `os.environ.get`?" — typed, validated, single object, and it errors at boot instead of at first use.

4.3 `backend/app/core/security.py` — auth crypto

Purpose: the four primitives of authentication. **Functions:** `hash_password` / `verify_password` (bcrypt via passlib) and `create_access_token` / `decode_access_token` (HS256 JWT via python-jose). **I/O:** plaintext ↔ hash; claims ↔ signed string. **Lifecycle:** called at login (hash verify + token mint) and on every authenticated request (decode). Fully dissected in §8.

4.4 `backend/app/db/session.py` — the connection factory

```
engine = create_engine(settings.database_url)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
def get_db() -> Generator:
    db = SessionLocal()
    try: yield db
    finally: db.close()
```

Purpose: one SQLAlchemy *engine* (a pool of DB connections) for the whole process; `get_db` hands each request its own *session* (a unit of work) and guarantees it closes. **Why `yield`:** FastAPI treats generator dependencies as setup/teardown — the `finally` runs after the response is sent, returning the connection to the pool. **Why `autocommit=False`:** services control transaction boundaries explicitly with `db.commit()`. **Interview Q:** "What happens if you forget to close sessions?" — connection-pool exhaustion; the app hangs waiting for a free connection.

4.5 `backend/app/domain/shift_coverage.py` — overnight-aware overlap

Purpose: answer "does a shift cover a coverage time-slot?", correctly handling shifts that wrap past midnight (21:00→06:00). **Key idea:** `shift_intervals` splits a wrapping shift into two same-day intervals `[(start,23:59:59),(00:00,end)]`; `intervals_overlap` is the classic `a_start < b_end AND b_start < a_end`; `shift_covers_slot` returns true if any piece overlaps. **Why pure & shared:** both the solver and Excel re-validation must agree on coverage or a manually-imported roster could pass one check and fail the other. **Interview Q:** "How do you test overnight shifts?" — unit tests feed `(21:00,06:00)` against morning/overnight slots, no DB needed.

4.6 `backend/app/domain/leave.py` — leave date expansion

Purpose: given a leave request type and dates, return the exact list of calendar dates it consumes. `leave_full` / `leave_half` → single date; `leave_multi` → every date in the inclusive range. Also exports the `LEAVE_TYPES` set used everywhere to branch leave vs. non-leave. **Pitfall it prevents:** off-by-one on the range (`(end-start).days + 1` is inclusive).

4.7 `backend/app/solver/model.py` — the pure CP-SAT model

Purpose: the mathematical heart. Takes plain dataclasses (`SolverInput`) and returns plain dataclasses (`SolverResult`) with *no* database or FastAPI imports — so it's unit-testable and reasoning about it is isolated from I/O. **Main dataclasses:** `SolverAgent, SolverShift, SolverCoverageRequirement, SolverRequest, SolverWeights, SolverInput` (in) and `SolverAssignment, SolverRequestDecision, SolverResult` (out). **Main function:** `solve(problem)`. Dissected line-by-line in §5. **Interview gold:** the module docstring is a spec of every hard vs soft constraint — read it aloud in an interview and you sound like you designed it (you did).

4.8 `backend/app/solver/service.py` — DB ↔ solver glue

Purpose: `generate_roster(db, week_cycle_id)` — load everything the solver needs from Postgres, translate ORM rows into solver dataclasses, run `solve`, then persist assignments, flip request statuses, sync leave balances, write conflict reports and satisfaction metrics, all in one transaction. **Callers:** the `POST /api/roster/generate` route and the auto-publish job. **Callees:** `solve`, `sync_leave_balance`, `get_or_create_solver_weights`. **Key subtlety:** it computes the first-come-first-served `rank_by_id` by sorting requests on `(created_at, id)` and scales float weights to integers (`_scale_weight`) because CP-SAT objective coefficients must be integers. Fully traced in §6.

4.9 `backend/app/services/leave_balance_sync.py` — delta accounting

Purpose: keep an agent's annual leave balance correct no matter how many times a roster is regenerated. **The clever bit:** it looks at the request's *previous* status and the *new* honored/denied outcome and applies only the **delta** — decrement when a request becomes honored, refund when it flips approved→denied, do nothing when unchanged. **Why delta not absolute:** regenerating a roster must be idempotent; naïve "subtract days on approve" would double-charge on the second run. **Interview Q:** "How do you avoid double-deducting leave when the manager regenerates?" — this function is the answer.

4.10 `backend/app/services/roster_lifecycle.py` — state machine + cron jobs

Purpose: enforce the roster/cycle state machine (`draft → published → locked`) and provide the scheduled `auto_publish_due_cycles` / `auto_lock_due_cycles`. **Guards:** only a draft can be published, only a published roster can be locked, a locked cycle rejects further changes. **Auto-publish rule (important):** it publishes a due cycle's roster *only if it has zero conflicts* — anything with unmet requests is held as a draft for human review. **Interview Q:** "How do you prevent an unreviewed bad roster from going live automatically?" — the zero-conflict gate.

4.11 `backend/app/services/roster_excel.py` — export / import / override

Purpose: managers live in Excel. This generates a styled workbook (`export_roster_workbook`), parses an uploaded one (`parse_import_file`), and — crucially — **re-validates every imported/overridden assignment against the same hard constraints the solver used** (`revalidate_and_apply_import`, `apply_manual_override`) before committing. If a human edit violates a hard rule (double-booking, skill mismatch, headcount cap), it's rejected with the exact violations. **Interview Q:** "A manager hand-edits the spreadsheet and breaks a rule — what stops that?" — re-validation reuses `domain/` logic so import and solver enforce identical rules.

4.12 The repetitive files (described once)

Every resource follows the same trio: `models/<x>.py` (table), `schemas/<x>.py` (Pydantic in/out), `crud/<x>.py` (queries), `api/routes/<x>.py` (endpoints). For example, *skills*: `Skill` model → `SkillCreate/SkillOut` schemas → `crud/skill.py` functions → `routes/skills.py` CRUD endpoints. Learn one and you've learned all thirteen. The **only** non-generic ones are the solver, lifecycle, excel, and leave-sync files above — spend your interview prep there.

4.13 Frontend key files

File	Purpose & interview hook
`src/api/client.ts`	Single axios instance. A **request interceptor** attaches the JWT from `localStorage`; a **response interceptor** catches 401 and force-logs-out. `extractErrorMessage` normalizes FastAPI's three error shapes (string, structured object with violations, Pydantic array) into one display string. *Q: where does the token live and what are the XSS implications?* (§8, §15).
`src/auth/AuthContext.tsx`	React Context storing token/role/agentId, initialised lazily from `localStorage`. `login` calls the API, then **decodes the JWT client-side** (base64 of the middle segment) to read `role` / `agent_id` for routing. *Q: is client-side decode a security risk?* No — it's only for UX routing; the server re-verifies the signature on every call. Never trust the client decode for authorization.
`src/components/ProtectedRoute.tsx`	A route guard: redirects to `/login` if unauthenticated, or to `/` if the role doesn't match. Purely a UX gate — real enforcement is server-side.
`src/App.tsx`	The whole route table: public `/`, `/login`, nested `/agent/*` and `/manager/*` trees wrapped in role guards, and a `/manager/config/*` sub-tree. Reading it top-to-bottom teaches you the entire app's navigation.
`src/components/RosterGrid.tsx` + `lib/rosterGrid.ts`	The visual schedule. The `lib` function is a **pure transform** turning a flat `RosterAssignment[]` into an agent×day grid; the component just renders it. Separation = the transform is testable without React.

What you should remember

There are only **~6 files that are truly unique**: `solver/model.py`, `solver/service.py`, `leave_balance_sync.py`, `roster_lifecycle.py`, `roster_excel.py`, and the two `domain/` files. Everything else is one CRUD pattern repeated. In an interview, steer toward those six.

5 · Every Function Explained — the Solver Core

This is the most important section, so we go deep on `solve()` in `solver/model.py`. First, the mental model, then a phase-by-phase walkthrough of the actual code.

5.1 What is CP-SAT, from zero?

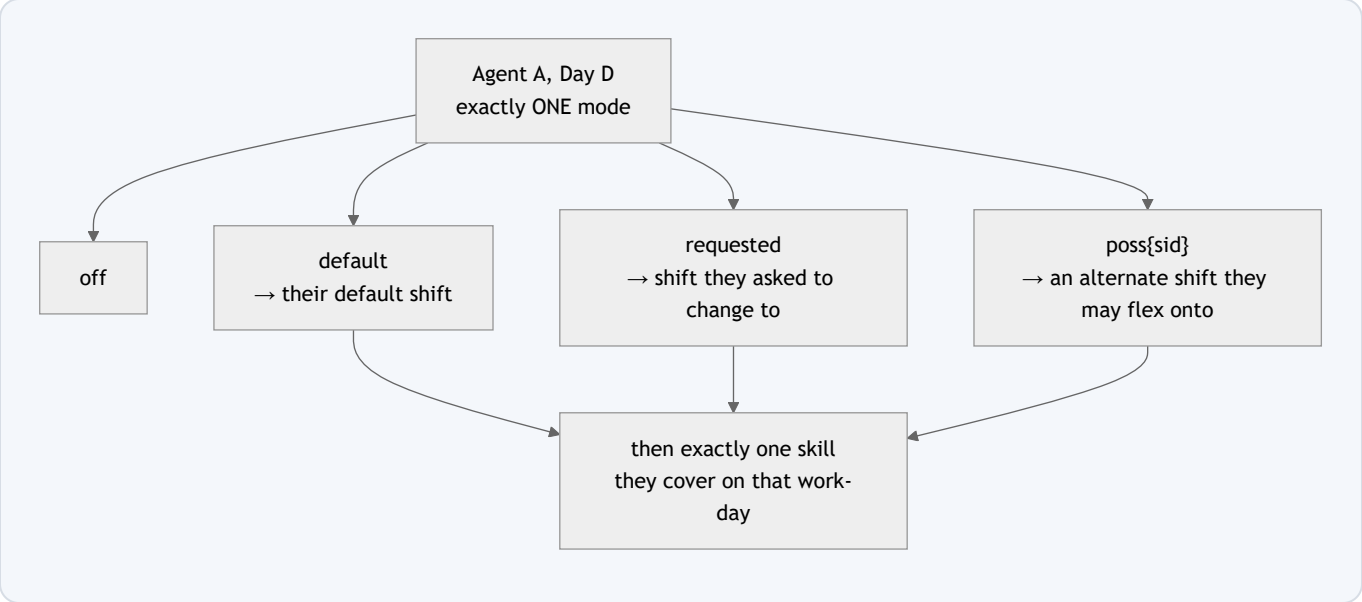
CP = Constraint Programming: you describe a problem as *variables* (unknowns), *constraints* (rules the unknowns must obey), and an *objective* (a number to minimize/maximize), then a *solver* finds values that satisfy all constraints and optimize the objective. **SAT** = Boolean satisfiability, the engine underneath. **Google OR-Tools' CP-SAT** is a world-class implementation. **Why use it here instead of hand-written loops?** Scheduling is combinatorial: with *n* agents × 7 days × several shift modes, the number of possible rosters explodes far beyond brute force, and greedy `if/else` heuristics get "stuck" (they can't backtrack to honour a later request by rearranging earlier ones). CP-SAT does global search with backtracking and proof of optimality.

The three building blocks in this model

- **BoolVar** — a 0/1 unknown. Here: "is agent A *off* on day D?", "does A work their *default* shift on D?".
- **Constraint** — e.g. `AddExactlyOne(modes)` : exactly one of {off, default, requested, possible-shifts} is true per agent per day (this *is* the no-double-booking rule).
- **Objective** — `Minimize(weighted sum of "denied" penalties)` : every unhonoured request adds its weight; the solver minimizes total pain.

5.2 The variable design (the key insight)

Instead of a variable per (agent, day, shift, skill) — which explodes — the model uses a compact **"mode" per agent per day**:




```
modes = {"off": model.NewBoolVar( ... )}
if agent.default_shift_id is not None:
    modes["default"] = model.NewBoolVar( ... )
if d in shift_change_dates and shift_change_dates[d] is not None:
    modes["requested"] = model.NewBoolVar( ... )
for sid in possible_ids:
    modes[f"poss{sid}"] = model.NewBoolVar( ... )
model.AddExactlyOne(modes.values())     # ← no double-booking, as one line
```

Then for each work-mode, exactly one skill is "covered": `model.Add(sum(skill_assigns.values()) == mode_var)` — meaning if the mode is chosen (1) exactly one skill assignment is 1; if not chosen (0) all skill assignments are 0.

5.3 `solve(problem: SolverInput) → SolverResult` — phase by phase

Parameters & return

In: `SolverInput` = the Monday week-start, lists of agents/shifts/coverage/requests, and the integer `SolverWeights`. **Out:** `SolverResult` = status (`optimal|feasible|infeasible`), the chosen `assignments`, and per-request `decisions` (honored true/false).

Phase 0 — Index the requests

Builds `requests_by_agent_date` (expanding leave into its covered dates via `leave_dates`) and `off_day_request_dates`. Skips `other` requests entirely (not modelled).

Phase 1 — Decision variables

The mode/skill variables from §5.2, per agent per day. **Complexity:** O(agents × 7 × modes × skills) variables.

Phase 2 — Coverage as a SOFT target (with a shortfall variable)

```
shortfall = model.NewIntVar(0, req.min_agents_required, ...)
model.Add(sum(terms) + shortfall ≥ req.min_agents_required)
objective_terms.append(problem.weights.coverage * shortfall)
```

Why a shortfall variable? If coverage were a hard `≥`, an understaffed week would make the whole model *infeasible* — the solver returns nothing and the manager is stuck. Instead, a `shortfall` "absorbs" missing agents and each missing head is penalized (weight 500, set *above* every request weight). Result: the solver meets coverage whenever possible, and only ever lets a slot go short as a last resort — degrading gracefully instead of crashing. **This is the single most important design decision in the model.**

Phase 3 — HARD weekly rest day

```
rest_pool = [d for d in week_dates if not any(leave on that day)]
rest_days_owed = min(agent.default_off_days_per_week, len(rest_pool))
model.Add(sum(off on rest_pool) == rest_days_owed)
```

Every agent takes exactly their owed rest days, chosen only from days they haven't taken leave (leave stacks *on top of* the guaranteed rest day, never consuming it). The `min()` prevents a fully-leave week from being infeasible. A **fixed** default off-day additionally pins the rest day to a specific weekday — unless an off-day request overrides the placement that week.

Phase 4 — HARD per-shift headcount cap

For each capped shift (e.g. Overnight `max_agents=2`) and each day: `sum(agents whose chosen mode resolves to that shift) ≤ max_agents`.

Phase 5 — SOFT request penalties + FCFS tie-breaker

For each request on each covered date, a `denied` BoolVar is tied to whether the wish came true (e.g. leave honored ⇔ agent is off ⇔ `denied == 1 - off_var`), and `weight × denied` is added to the objective. Simultaneously an FCFS term `protection × denied` is accumulated, where `protection = num_requests - submitted_rank` (earlier submissions cost more to deny).

Phase 6 — Fairness (minimax)

```
max_denied = model.NewIntVar(0, len+1, "max_denied")
for agent: model.Add(max_denied ≥ sum(that agent's denials))
objective_terms.append(fairness * max_denied)
```

Why minimize the maximum, not the total? Minimizing total denials could dump *all* denials on one unlucky agent. Minimax spreads the pain — it minimizes the worst individual experience. Classic fairness formulation.

Phase 7 — The lexicographic objective (the subtle masterpiece)

```
model.Minimize(sum(objective_terms) * (fcfs_cap + 1) + sum(fcfs_terms))
```

The problem: we want FCFS to break *ties* but never override a real trade-off. **The solution:** scale the main objective by `(fcfs_cap + 1)`, where `fcfs_cap` is the largest the entire FCFS term can ever reach. Now any genuine improvement in the main objective (≥1 before scaling) is worth more than the whole FCFS term combined — so FCFS can only ever settle exact ties, deterministically preferring the earlier request. This is **lexicographic optimization via magnitude separation**, and it's a fantastic thing to explain in an interview.

Phase 8 — Solve & extract

```
solver.parameters.max_time_in_seconds = 30
status = solver.Solve(model)
if status not in (OPTIMAL, FEASIBLE): return SolverResult(status="infeasible")
```

Then it reads back the chosen assignments and computes each request's `honored` = *all* its condition vars are true (matters for multi-day leave spanning several off-vars).

5.4 Complexity, edge cases, failures

Aspect	Detail
Time complexity	NP-hard in general; CP-SAT is worst-case exponential but empirically fast for this size. Bounded by the 30-second cap — returns best-found feasible solution if not proven optimal.
Space complexity	O(agents × days × modes × skills) variables + constraints; kilobytes for a real team.
Edge: no default shift	Agent can only be "off"; rest-day quota is skipped (meaningless).
Edge: fully-leave week	`min(owed, len(rest_pool))` keeps it feasible.
Edge: overnight coverage	Handled by `shift_covers_slot` interval-splitting.
Failure: truly infeasible hard set	e.g. a required skill nobody holds → status infeasible → service raises 422 with a plain-English reason.

5.5 Alternatives considered

Alternative	Why not chosen
Greedy heuristic (sort requests, assign one by one)	Can't backtrack; produces unfair, sub-optimal rosters; "first agent gets everything".
Integer Linear Programming (e.g. PuLP/CBC)	Works, but CP-SAT is faster on Boolean-heavy scheduling and has cleaner primitives (`AddExactlyOne` , reification).
Genetic algorithm / simulated annealing	No optimality proof, harder to guarantee hard constraints, non-deterministic.
Hand-written rules engine	Unmaintainable as constraints grow; every new rule risks breaking others.

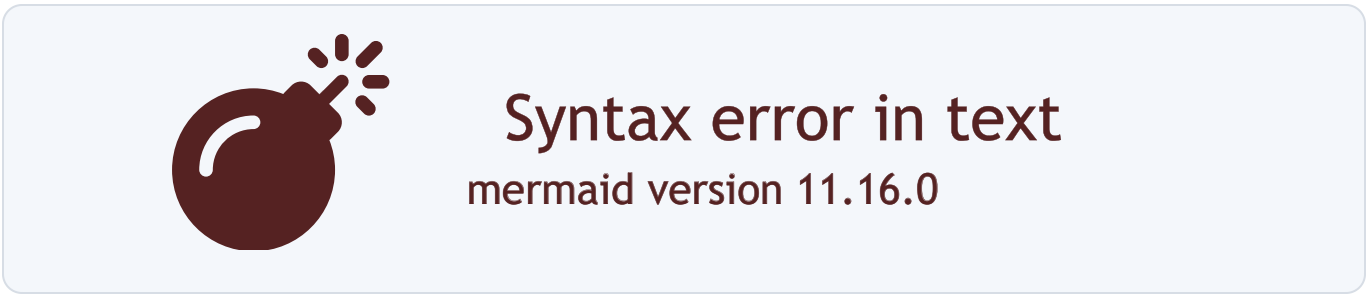
What you should remember

Three sentences win the solver conversation: **(1)** "Hard rules are constraints, soft wishes are weighted penalties in a minimize objective." **(2)** "Coverage is soft with a shortfall variable so the model can never be made infeasible by understaffing — it degrades gracefully." **(3)** "Fairness is minimax and FCFS is a lexicographic tie-breaker scaled below the main objective so it can never override a real decision."

Simple analogy: the objective is a golf score — every denied wish adds strokes. Coverage denials cost a lot, leave a medium amount, shift-changes a little. The solver plays the whole course to card the lowest total, then uses "who asked first" only to break a literal tie.

6 · Complete Request Flow — "Manager clicks *Generate Roster*"

We trace one request through **every** component, because it exercises auth, routing, the solver, transactions, and multiple tables.



Step-by-step narration

1. **Frontend trigger:** the manager clicks "Generate" in `RosterWorkspace.tsx`; a React Query mutation calls `generateRoster(weekCycleId)` from `endpoints.ts`.
2. **axios interceptor** attaches `Authorization: Bearer <jwt>` from localStorage and sends `POST /api/roster/generate?week_cycle_id=5`.
3. **CORS middleware** checks the Origin against the allow-list; the router matches the path.
4. **Dependency** `require_manager` runs before the endpoint: `get_current_user` decodes the JWT (`decode_access_token`), loads the `User`, checks `active`; then asserts `role == manager` (else 403). The router also has a router-level `dependencies=[Depends(require_manager)]`, so this is enforced twice by design.
5. **Endpoint** `generate()` calls `generate_roster(db, 5)` inside a try/except that maps `RosterGenerationError` to the right HTTP status.
6. **Service loads state** with eager-loaded relationships (`joinedload(Agent.skill_links, Agent.possible_shift_links)`) to avoid N+1 queries, filters requests to `pending|approved` and non-`other`, and fetches the singleton solver weights.
7. **Build & scale:** requests are ranked FCFS; float weights are multiplied by 100 (`_scale_weight`) because CP-SAT needs integers.
8. **Solve** (§5). If infeasible → 422 with a human reason.
9. **Persist in one transaction:** insert `Roster` (flush to get its id) → insert every `RosterAssignment` → for each request decision, flip status, delta-sync leave balance, insert a `ConflictReport` if denied → insert per-agent and aggregate `SatisfactionMetric` → `db.commit()`.
10. **Audit:** the route records an audit entry (`roster_generated`) and commits.
11. **Response:** a 201 with roster + assignments + conflicts + satisfaction; React Query caches it and the grid, conflict list, and satisfaction bars render.

Common confusion to preempt

Two commits happen — one inside the service (roster + assignments + balances) and one in the route (audit). An interviewer may ask "isn't that two transactions; what if the audit fails?" Honest answer: yes, they're separate; the audit is best-effort metadata, so a rare audit failure doesn't roll back a valid roster. A stricter design would wrap both in one transaction — a fair "future improvement" to volunteer.

7 · Database

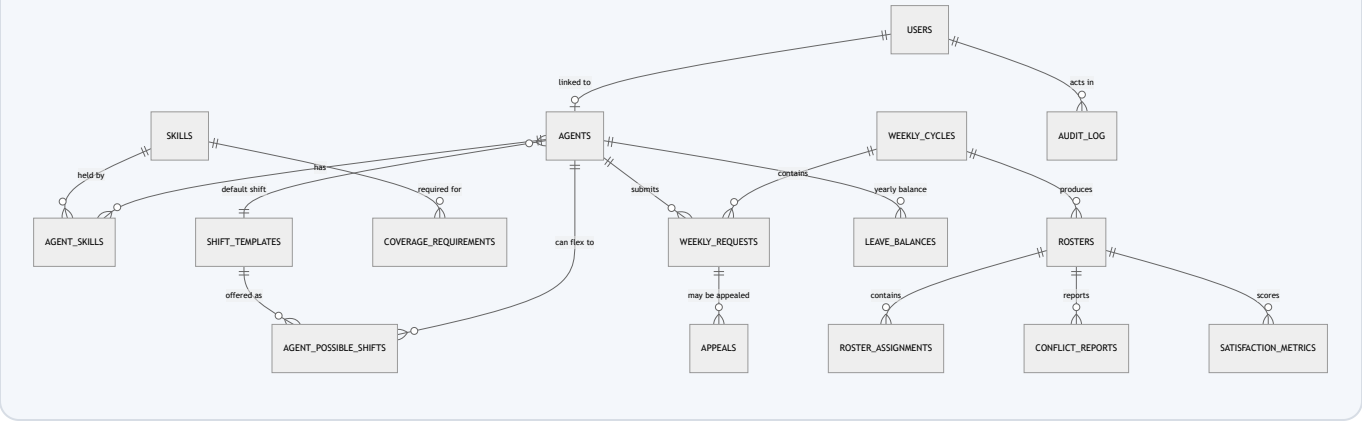
7.1 Why PostgreSQL (relational)?

The domain is a web of relationships with strict integrity needs: an assignment must reference a real agent, shift, and skill; a roster's rows must all commit or none; a leave balance has a uniqueness rule (one row per agent per year). Relational databases give **foreign keys** (referential integrity), **transactions** (all-or-nothing writes — ACID), and **constraints** (uniqueness). A document DB (MongoDB) would push all that integrity into application code. Postgres specifically: free, rock-solid, great typing, and Supabase offers managed Postgres on a free tier.

ACID, unpacked

Atomicity (all-or-nothing), **C**onsistency (constraints always hold), **I**solation (concurrent transactions don't corrupt each other), **D**urability (committed = survives a crash). Roster generation relies on Atomicity: if anything fails mid-write, the half-built roster vanishes on rollback.

7.2 Schema (entity-relationship)



7.3 Why each table exists

Table	Reason to exist / notable fields
`users`	Login identity + role. `agent_id` links a login to an agent profile (managers may have none). `hashed_password` never stores plaintext.
`agents`	The schedulable person. `default_shift_id`, `default_off_day_type` (fixed/flexible), `default_off_day`, `default_off_days_per_week` drive the solver's rest-day logic.
`skills`, `agent_skills`	Many-to-many: an agent holds several skills; a skill is held by many. Coverage is expressed per skill.
`agent_possible_shifts`	Alternate shifts an agent may flex onto (e.g. cover Overnight 1–2 days). Feeds the solver's `poss{sid}` modes.
`shift_templates`	Named time windows. `max_agents` is the hard per-day headcount cap; `end<start` encodes overnight shifts.
`coverage_requirements`	Per weekday + time-slot + skill minimum staffing. `is_peak` / `weight` for future prioritization.
`weekly_cycles`	The unit of scheduling. Holds the four deadline timestamps (request, publish, appeal, lock) and status. `week_start_date` is unique — one cycle per week.
`weekly_requests`	An agent's ask for a given cycle. Type, dates, optional shift/half-day, status, `denial_reason`, `submitted_via` (form vs ai_parsed), `created_at` (drives FCFS).
`rosters`, `roster_assignments`	The output. A roster has many assignments (agent/date/shift/skill/source). `source` distinguishes solver vs manual override.
`conflict_reports`	Human-readable "why this request wasn't honored", with severity — the notification surface.
`satisfaction_metrics`	Per-agent and aggregate honored-request percentage — the fairness/quality dashboard.
`appeals`	An agent's challenge to a denied request; manager resolves with response + timestamp.
`leave_balances`	Annual allotment/taken/remaining per agent per year. `UniqueConstraint(agent_id, year)` prevents duplicates.
`solver_weights`	Singleton (id=1) of tunable soft-constraint weights — lets managers reshape solver behavior without code.

`audit_log`	Append-only trail: actor, action, target, old/new value, reason, timestamp. `actor_id` null = system action.

7.4 Normalization, indexes, constraints, concurrency

- **Normalization:** the schema is ~3NF — no repeating groups, join tables for many-to-many (`agent_skills`), no duplicated agent data across rosters. This avoids update anomalies (change a skill name once, not per assignment).
- **Denormalization:** deliberately minimal. `satisfaction_metrics` is a mild denormalization — a precomputed percentage stored rather than recomputed on read — justified because it's a snapshot of a specific roster.
- **Indexes:** primary keys are indexed automatically; unique constraints (`users.email`, `skills.name`, `weekly_cycles.week_start_date`, `leave_balances(agent_id,year)`) create unique indexes. *Interview-honest note:* the foreign-key columns queried hottest (e.g. `roster_assignments.roster_id`, `weekly_requests.week_cycle_id`) would benefit from explicit indexes — a concrete, easy optimization to name (§13/§19).
- **Constraints:** foreign keys everywhere; NOT NULL on required fields; unique constraints as above; enums enforced at the DB level (`Enum( ... , name="request_type")`).
- **Transactions & locks:** `generate_roster` does its whole write in one transaction. SQLAlchemy sessions use Postgres' default *read-committed* isolation. There's no explicit row-locking; the app's write pattern (one manager generating a roster) has low contention. Under high concurrency you'd add optimistic locking / `SELECT … FOR UPDATE` (§14).
- **Connection pooling:** the single `create_engine` maintains a pool; each request borrows and returns a connection via `get_db`. On Supabase, the docs specify the *Session* pooler (port 5432) because migrations and long-lived connections need session mode.

7.5 Migration strategy (Alembic)

What Alembic is: version control for schema. Each file in `alembic/versions/` has `upgrade()` / `downgrade()` and a parent revision, forming a chain. History here: initial schema → add solver-weights table → add agent-possible-shifts table → add shift `max_agents` → drop a default-off-day weight. **Why not autocreate tables?** `Base.metadata.create_all` can't evolve an existing DB safely; migrations give reproducible, reversible, reviewable changes and are run once on deploy.

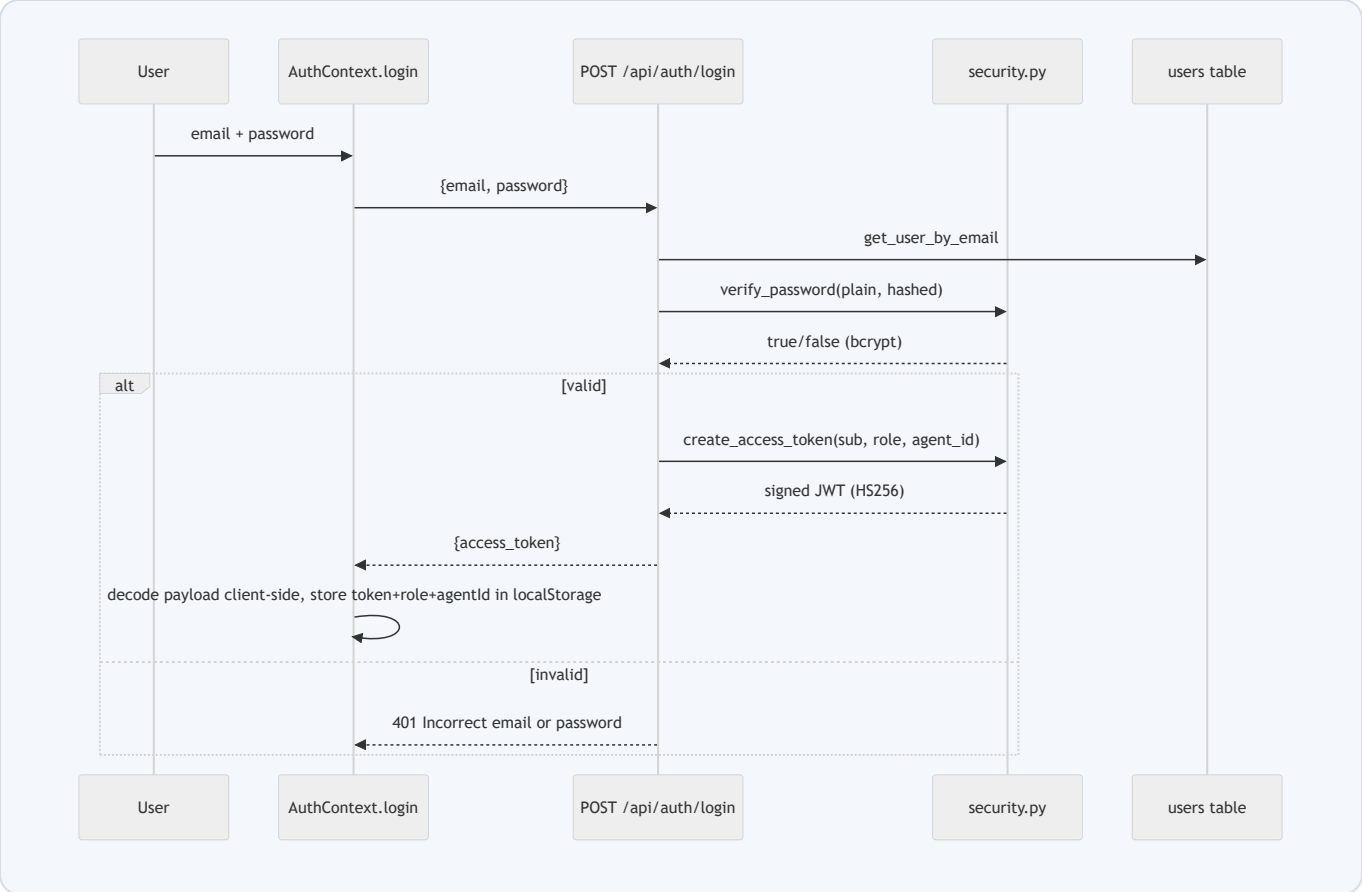
What you should remember

Relational + transactional because roster writes are multi-row and must be atomic. The schema is normalized with join tables for the many-to-manys. The two constraints worth quoting: `UniqueConstraint(agent_id, year)` on balances and unique `week_start_date` on cycles. The honest optimization to volunteer: add indexes on hot foreign keys.

Analogy: tables are labelled filing cabinets, foreign keys are cross-reference stickers, transactions are "staple these five forms together or file none of them".

8 · Authentication & Authorization

8.1 How login works (end to end)



8.2 The JWT lifecycle

What a JWT is: three base64url segments `header.payload.signature`. The payload here carries `sub` (user id), `role`, `agent_id`, and `exp` (expiry). The signature is `HMAC-SHA256(header.payload, jwt_secret_key)` — only someone with the secret can produce it, so the server can trust an unmodified token without a database lookup for the token itself.

```
def create_access_token(*, subject, role, agent_id):
    expire = datetime.now(timezone.utc) + timedelta(minutes=settings.jwt_expire_minutes)  #
    payload = {"sub": subject, "role": role, "agent_id": agent_id, "exp": expire}
    return jwt.encode(payload, settings.jwt_secret_key, algorithm="HS256")

def decode_access_token(token):
    try: return jwt.decode(token, settings.jwt_secret_key, algorithms=["HS256"])
    except JWTError: raise ValueError("Invalid or expired token")
```

On every request: `get_current_user` (deps.py) extracts the bearer token, decodes+verifies it (signature and `exp` checked by python-jose), reads `sub`, loads the `User`, and rejects if the user is missing or inactive. Then `require_manager` / `require_agent` enforce the role.

8.3 Refresh tokens, sessions, cookies

Honest scope statement: this project uses a **single 8-hour access token, no refresh token, and no server-side session**. The token is stored in `localStorage`, not a cookie. That's a deliberate simplification for the product's scale (small team, short shifts). The trade-offs, and how you'd evolve it, are in §15/§19.

8.4 Authorization / RBAC

RBAC = Role-Based Access Control: permissions attach to roles, not individuals. Two roles: `manager` and `agent`. Enforcement is layered: (1) route-level `dependencies=[Depends(require_manager)]` on whole routers (e.g. all of `/api/roster`), (2) per-endpoint guards, and (3) ownership checks (an agent's `/mine` endpoints filter by *their* `agent_id` from the token, so one agent can't read another's data).

8.5 Threats & mitigations

Threat	What it is	Mitigation here
Password theft	DB leak exposes passwords	bcrypt (salted, slow) — never plaintext; passlib upgrades hashes over time.
Token tampering	User edits their JWT to become manager	HMAC signature check fails → 401. You can't forge without the secret.
Replay attack	Stolen token reused	`exp` limits the window to 8h. (No revocation list — a known gap; §15.)
SQL injection	Malicious input as SQL	SQLAlchemy parameterizes all queries — inputs are never string-concatenated into SQL.
XSS	Injected script reads localStorage token	React escapes rendered content by default; no `dangerouslySetInnerHTML`. (localStorage is still JS-reachable — see §15.)
CSRF	Another site makes authed requests using your cookie	Largely N/A: auth is a bearer header, not a cookie, so a cross-site form can't attach it automatically.
Privilege escalation	Agent hits a manager route	`require_manager` returns 403; client-side role check is only cosmetic.

What you should remember

Login = verify bcrypt hash, mint an HS256 JWT carrying `sub/role/agent_id/exp`. Every request re-verifies the signature and role server-side; the client-side decode is *only* for routing UX. The one line that impresses: "*authorization is enforced on the server on every request — the frontend guard is convenience, not security.*"

Common confusion: "the frontend checks the role, so it's secure." No — anyone can call the API directly; security is the server-side `require_manager` dependency.

9 · APIs

All endpoints are under `/api`. Auth column: 🌐 public · 👤 any logged-in · 👨‍💼 manager only. Full inventory:

Method + Path	Auth	Purpose
POST /auth/login	🌐	Exchange credentials for a JWT.
POST /auth/users	👨‍💼	Create a user account (manager provisions logins).
GET /roster/current	🌐	Public published roster for the current week.
GET /roster/{week_start_date}	🌐	Public roster for a given week.
GET / POST / PATCH / DEL /skills	👤 / 👨‍💼	CRUD skills.
… /shift-templates	👤 / 👨‍💼	CRUD shifts (read any logged-in; write manager).
… /coverage-requirements	👨‍💼	CRUD coverage minimums.
… /agents	👨‍💼	CRUD agents (+ skills, possible shifts, off-day config).
… /leave-balances	👤 / 👨‍💼	Agents read `/mine`; managers CRUD any.
GET /weekly-cycles, /current, POST	👤 / 👨‍💼	List/read cycles; create (manager).
POST /requests, GET /mine, GET "", PATCH /{id}	👤 / 👨‍💼	Submit request; list mine; list all + review (manager).
GET / PATCH /solver-config	👨‍💼	Read/tune solver weights.
POST /roster/generate	👨‍💼	Run the solver, persist a draft roster.
GET /roster/{id}/detail·assignments·conflicts·satisfaction	👨‍💼	Read a roster and its analytics.
POST /roster/{id}/publish · /lock	👨‍💼	Advance the state machine.
GET /roster/{id}/export · POST /import · /override	👨‍💼	Excel round-trip & manual override (re-validated).
… /appeals (POST, /mine, "", PATCH)	👤 / 👨‍💼	Agents file/read appeals; managers list/resolve.
GET /audit, /audit/mine	👤 / 👨‍💼	Manager sees all; agent sees their own trail.

GET /health	🌐	Liveness probe.

Anatomy of one endpoint (the pattern for all)

Take `PATCH /api/requests/{id}` (review a request): **Request** = path id + `WeeklyRequestReview` body (validated by Pydantic). **Auth** = `require_manager`. **Business logic** = load request (404 if missing) → capture old status → `crud.review_request` → write an audit row → commit. **Response** = `WeeklyRequestOut`. **Status codes** = 200 ok, 404 not found, 400 domain error, 401 no token, 403 wrong role, 422 validation. **Optimization** = single query load, single commit. Every route mirrors this: *validate → authorize → delegate to crud/service → audit → serialize*.

What you should remember

The API is RESTful and resource-oriented: nouns as paths, HTTP verbs as actions, Pydantic `response_model` on everything for typed output, and consistent status codes. Public roster reads need no auth by design — that's the "anyone can see the schedule" requirement.

10 · Frontend

10.1 Routing

`react-router-dom` v7 with nested routes in `App.tsx`. A shared `Layout` wraps everything; `ProtectedRoute role="…"` gates the `/agent` and `/manager` subtrees; `/manager/config` nests its own child routes rendered into an `<Outlet/>`. Vercel's `vercel.json` rewrite sends all non-`/api` paths to `index.html` so client-side routing works on refresh/deep-link.

10.2 State management — the key decision

There is **no Redux**. State is split by nature:

- **Server state** (data owned by the backend) → **TanStack React Query**. It caches responses, dedupes concurrent requests, refetches on invalidation, and exposes `isLoading/isError`. Mutations (generate, review, publish) call `queryClient.invalidateQueries` to refresh dependent views.
- **Auth state** (token/role) → a small React **Context** (`AuthContext`), persisted to `localStorage`.
- **Local UI state** (form fields, modals) → component `useState` + `react-hook-form`.

> **Why React Query instead of Redux?**
>
> Most "global state" in CRUD apps is really *cached server data*. Redux forces you to hand-write fetching, caching, and invalidation. React Query gives all three for free, so the app needs no global client store. Interview line: "*I separated server state from client state; server state is a cache, and React Query is a purpose-built cache.*"

10.3 Data fetching, caching, forms, validation

- **Fetching:** typed functions in `endpoints.ts` wrap the shared axios client; components call them inside `useQuery` / `useMutation`.
- **Caching:** React Query keeps results by query key; navigating back is instant from cache, then revalidated.
- **Forms:** `react-hook-form` uses uncontrolled inputs (refs) so typing doesn't re-render the whole form — performant by default; validation rules are declared per field.
- **Error display:** `extractErrorMessage` flattens FastAPI's varied error shapes into one string.

10.4 Rendering, performance, PWA

- **Pure transforms** (`lib/rosterGrid.ts`) keep components thin and testable.
- **PWA:** `vite-plugin-pwa` with `registerType:'autoUpdate'` generates a service worker + manifest, making the app installable and offline-capable for the shell.
- **Code splitting:** Vite/Rollup splits the bundle; route-level lazy loading is the obvious next optimization (§13).
- **Memoization:** `useMemo` in `AuthContext` stabilizes the context value so consumers don't re-render on every parent render.

What you should remember

The frontend's headline decision is **"server state ≠ client state"**. React Query owns the former; a tiny Context owns auth; components own trivial UI state. That single distinction is why there's no bulky global store, and it's the most likely frontend interview topic.

11 · Backend

11.1 The layering, named in industry terms

Layer	This project	Responsibility
Controller	`api/routes/*`	HTTP parse, authorize, delegate, serialize. No business logic.
Service	`services/*`, `solver/service.py`	Multi-step orchestration across models + external logic (solver).
Repository	`crud/*`	The only place issuing DB queries per resource.
Domain	`domain/*`, `solver/model.py`	Pure business rules, no I/O.
Model	`models/*`	Persistence mapping.
Schema	`schemas/*`	Edge validation/serialization.

11.2 Dependency Injection

What DI is: instead of a function creating its own dependencies, they're *passed in*. FastAPI's `Depends()` is DI: `db: Session = Depends(get_db)` and `current_user = Depends(require_manager)` mean the framework builds and injects those before your endpoint runs. **Why it matters:** testability (inject a fake DB/user), reuse (auth logic written once), and clean signatures. In tests, `conftest.py` overrides `get_db` to point at the test database.

11.3 Middleware, validation, errors, logging, config

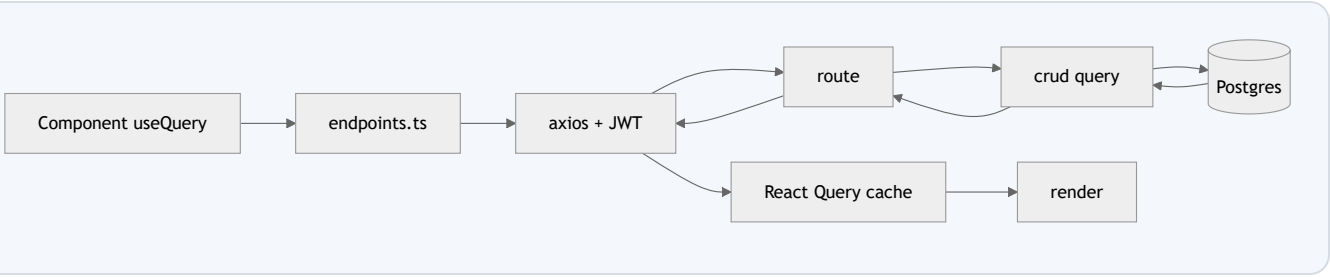
- **Middleware:** only CORS is registered globally; auth is per-route via dependencies (more granular than a blanket middleware).
- **Validation:** Pydantic schemas reject malformed bodies with 422 automatically, before your code runs.
- **Error handling:** domain errors (`RosterGenerationError`, `WeeklyRequestError`, `RosterImportError`, `RosterLifecycleError`) carry a `status_code` + `detail`; routes translate them into `HTTPException`. Import/override errors also return a structured `violations` list.
- **Configuration:** all secrets/URLs via env → `Settings`. Nothing hard-coded.
- **Logging:** currently relies on Uvicorn/Cloud Run request logs; structured app logging + tracing is a named future improvement (§18/§19).

What you should remember

Thin controllers, fat services, pure domain, repository-only DB access. DI via `Depends` is what makes auth reusable and the whole thing testable. If asked "how is this maintainable?", the answer is the strict layering — each layer has exactly one reason to change.

12 · Data Flow

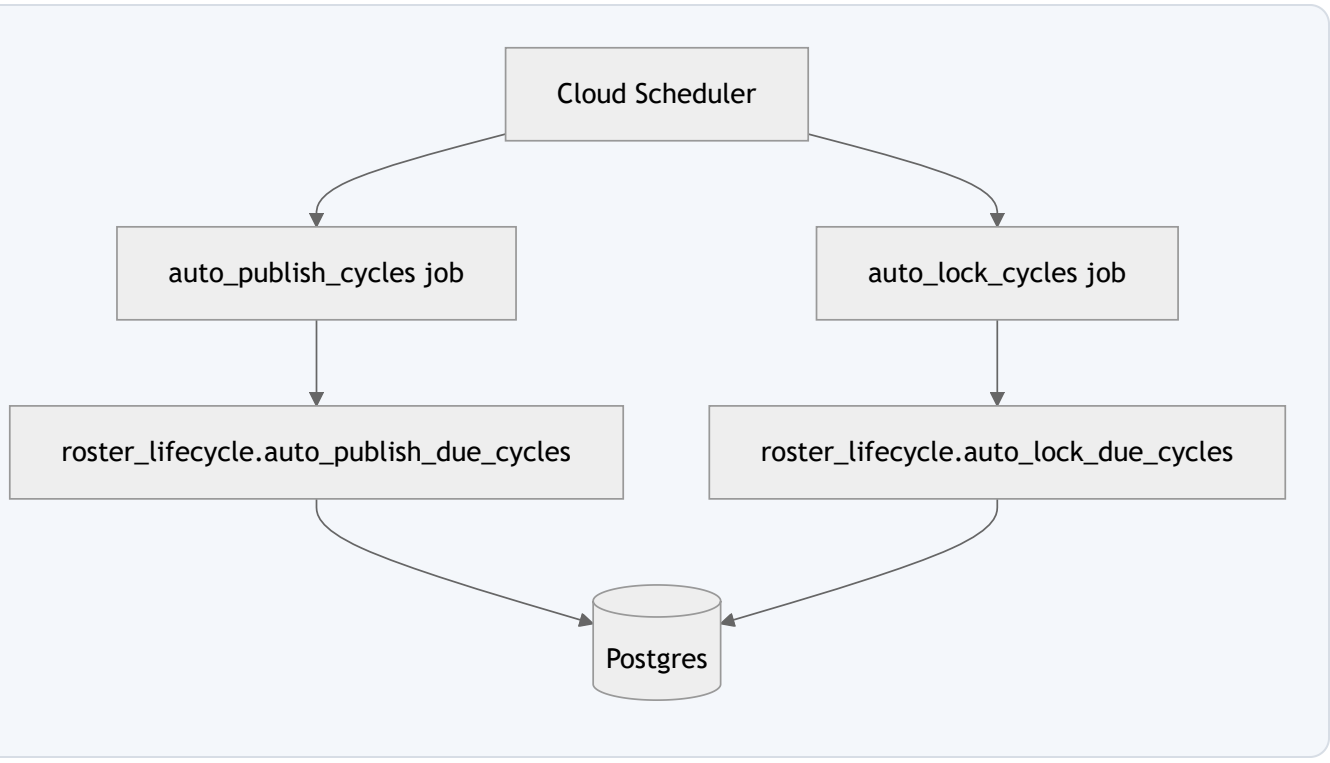
12.1 Read path (agent views their roster)



12.2 Write path with state change (generate roster)

Covered fully in §6. The data transformation chain: *ORM rows → solver dataclasses → CP-SAT variables → solution → ORM rows (assignments, statuses, balances, conflicts, metrics) → Pydantic JSON → React Query cache → grid*.

12.3 Background jobs



Publish rule: Saturday 00:00, publish only conflict-free rosters; hold the rest as drafts. **Lock rule:** Monday 00:00, lock cycle + its published roster. Both idempotent (safe to re-run).

12.4 Real-time?

None. There are no WebSockets/SSE; the UI is request-response with React Query revalidation. Real-time roster updates would be a future feature (§19).

13 · Optimization Techniques

Technique	Where / How	Why & trade-off
Eager loading (N+1 fix)	`joinedload(Agent.skill_links, Agent.possible_shift_links)` in `generate_roster`	Loads agents+skills+shifts in one query instead of one-per-agent. Trade-off: a slightly larger single query vs many round trips.
Solver time cap	`max_time_in_seconds = 30`	Bounds worst-case latency; returns best feasible if not proven optimal. Trade-off: near-optimal, not always optimal, on hard weeks.
Integer weight scaling	`_scale_weight(x)=int(x*100)`	CP-SAT needs integer coefficients; ×100 preserves 2-decimal precision.
Compact variable model	"mode per agent-day" not per shift	Fewer variables → faster solve; the `AddExactlyOne` encodes no-double-booking for free.
Lexicographic scaling	objective ×(fcfs_cap+1)	Encodes a strict priority (FCFS never overrides real trade-offs) in a single objective — cheaper than solving twice.
Delta balance sync	`sync_leave_balance`	Only applies the change since last state → idempotent regeneration, no double-charge.
Client caching	React Query	Instant navigation, fewer requests; trade-off: staleness handled via invalidation.
In-memory Excel streaming	`BytesIO` + `StreamingResponse`	No temp files/storage; trade-off: memory-bound for huge exports.
Connection pooling	single engine	Reuses TCP+auth handshakes; trade-off: pool size tuning under load.
Uncontrolled forms	react-hook-form	Fewer re-renders while typing.

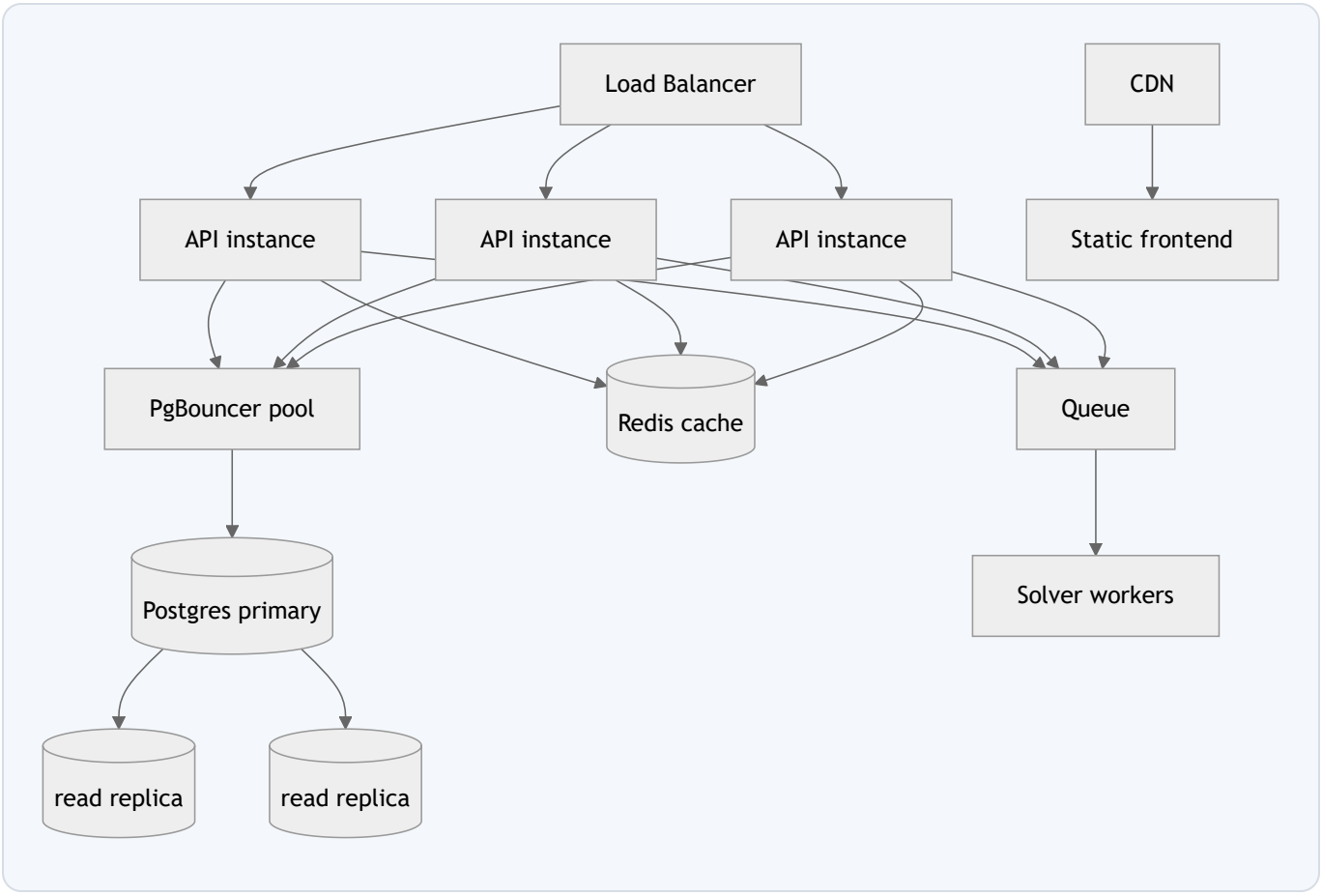
Named-but-not-yet-applied (honest, and good to volunteer)

Explicit indexes on hot FKs, pagination on list endpoints, route-level lazy loading/code-splitting, response compression (gzip/brotli), and rate limiting. Each is a one-line "here's exactly what I'd add and

when" in an interview.

14 · Scalability — what breaks first, at each scale

Scale	What breaks first	Fix
10 users	Nothing. Cloud Run scale-to-zero + free Postgres is plenty.	—
100 users	Still fine. Maybe cold-start latency on Cloud Run.	Min-instances=1 to kill cold starts.
1,000 users	Unindexed list queries slow; more agents = longer solves.	Add FK indexes; paginate; keep solve per-team not global.
10,000 users	DB connections + write contention; single solve blocks a worker for up to 30s.	Connection pooler (PgBouncer/Supabase pooler); move solve to a queue + worker pool; add Redis cache for public roster reads.
1,000,000 users	Single Postgres write ceiling; one region latency.	Read replicas; partition/shard by organization (multi-tenant); CDN for the SPA; autoscaled stateless API behind a load balancer; regional deployments.



Why the design already scales horizontally: the API is *stateless* (JWT, no server session) so you can run N identical instances behind a load balancer with zero coordination. The natural sharding key is the **organization/tenant** — this app is single-tenant today; multi-tenancy is the biggest architectural leap (§19).

15 · Security

Concern	Decision in this codebase
Authentication	bcrypt-hashed passwords; HS256 JWT bearer tokens with 8h expiry.
Authorization	Server-side RBAC via `require_manager/require_agent`; ownership filtering on `/mine` routes.
Transport encryption	HTTPS/TLS terminated by Cloud Run & Vercel — tokens never travel in clear.
Secrets	`jwt_secret_key`, `database_url` from env only; never committed. Cloud Run env vars / Secret Manager in prod.
Input validation	Pydantic on every request body/query; rejects bad types before logic runs.
SQL injection	SQLAlchemy parameterized queries — no string-built SQL anywhere.
XSS	React auto-escapes; no raw HTML injection.
CSRF	Bearer-header auth (not cookies) means cross-site requests can't ride along.
CORS	Explicit origin allow-list (not `*`) with credentials.
File upload	Excel read into memory, parsed with openpyxl, and **every row re-validated against hard constraints** before commit — a bad file can't corrupt the roster.
Audit	Append-only `audit_log` records every sensitive mutation with actor + old/new value.

Known gaps to own honestly (maturity signal in interviews)

- **Token in localStorage** → reachable by any XSS. Mitigation path: httpOnly cookie + CSRF token, or short access token + refresh rotation.
- **No token revocation** → a stolen token is valid until `exp`. Add a denylist or short-lived tokens + refresh.
- **No rate limiting** → login is brute-forceable. Add per-IP throttling / lockout.
- **Two-commit generate flow** (roster vs audit) — could be one transaction.

Naming your own gaps *with* the fix is exactly what separates a senior answer from a junior one.

16 · Design Decisions

Decision	Chosen	Rejected & why	Trade-off
Scheduling engine	OR-Tools CP-SAT	Hand heuristics (unfair/stuck), ILP (slower here), GA (no optimality)	Learning curve vs global optimality.
Coverage constraint	Soft w/ shortfall var	Hard $\geq$ (makes model infeasible when understaffed)	May under-cover as last resort vs never crashing.
Fairness	Minimax denials	Minimize total (dumps pain on one agent)	Slightly higher total denials, far fairer spread.
Tie-break	Lexicographic FCFS	Arbitrary/random tie-break	Extra scaling math vs deterministic fairness.
Auth	Stateless JWT	Server sessions (need shared store to scale)	No instant revocation vs free horizontal scaling.
Client state	React Query	Redux (boilerplate for cache)	Less "one store" mental model vs far less code.
Backend	FastAPI	Django (heavier), Flask (less async/validation)	Younger ecosystem vs speed + Pydantic + OpenAPI.
Purity split	Pure solver/domain	DB access inside solver (untestable)	Extra mapping code vs unit-testable core.
Deploy	Cloud Run + Supabase + Vercel	Always-on VM (costs money idle)	Cold starts vs ~$0 scale-to-zero.

17 · Technology Deep Dive

FastAPI

What: a modern Python web framework built on Starlette (ASGI) + Pydantic. **How it works:** type hints drive automatic request validation, dependency injection, and OpenAPI docs; async endpoints run on an event loop. **Here:** every route, the `Depends` auth, the Pydantic schemas. **Misconception:** "async makes everything faster" — CPU-bound work (the solve) still blocks the worker; async helps I/O concurrency. **Alternatives:** Django REST, Flask, Express. **Pros:** speed, validation, docs. **Cons:** younger than Django, fewer batteries.

Google OR-Tools CP-SAT

What: a constraint/SAT solver. **Internals:** lazy-clause-generation SAT engine with linear/integer reasoning, presolve, and parallel search; proves optimality or returns best-found within the time budget. **Here:** the entire `solve()`. **Misconception:** "it's AI/ML" — no, it's exact combinatorial optimization, deterministic and provable. **Alternatives:** Gurobi/CPLEX (commercial), PuLP+CBC, custom heuristics. **Pros:** free, fast, expressive. **Cons:** modelling skill required; large models can be slow.

SQLAlchemy 2.0 + Alembic

What: a Python ORM (object-relational mapper) + migration tool. **Internals:** the *Unit of Work* pattern — a session tracks changes and flushes them as SQL on commit; the *Identity Map* ensures one Python object per row. **Here:** typed `Mapped[...]` models, relationships, `joinedload` for eager loading. **Misconception:** "ORMs are always slow" — the N+1 problem is the usual culprit, avoided here with eager loading. **Alternatives:** raw SQL, Django ORM, Tortoise. **Pros:** safety, portability. **Cons:** abstraction can hide expensive queries.

JWT (python-jose) + bcrypt (passlib)

JWT internals: base64url header+payload signed with HMAC-SHA256; verification recomputes the HMAC with the secret. **bcrypt:** a deliberately slow, salted hash (work factor) so brute-forcing stolen hashes is costly. **Misconception:** "JWTs are encrypted" — they're *signed, not encrypted*; anyone can read the payload, so never put secrets in it. **Alternatives:** opaque session tokens, PASETO, Argon2 (for hashing).

React 19 + TypeScript + Vite + TanStack Query

React: component model with a virtual DOM diff; hooks for state/effects. **TypeScript:** compile-time types catch shape bugs. **Vite:** native-ESM dev server (instant HMR) + Rollup production builds. **React Query:** a server-state cache with keys, staleness, and invalidation. **Misconception:** "React Query is a state manager" — it's a *server-cache*; client state still needs Context/useState.

Cloud Run + Supabase + Vercel

Cloud Run: serverless containers, scale-to-zero, pay-per-request, HTTPS + autoscaling built in. **Supabase:** managed Postgres (+ auth/storage we don't use). **Vercel:** global static/edge hosting for the SPA with SPA rewrites. **Why this trio:** ~$0 for a demo, each best-in-class at its job, all managed.

18 · Common Bugs & Debugging

Symptom	Likely cause	How to debug
"CORS error" in browser	Frontend origin not in allow-list, or hitting wrong base URL	Check `cors_allowed_origins` env + `VITE_API_BASE_URL`; inspect the failed preflight in Network tab.
Generate returns 422 "no feasible roster"	A required skill nobody active holds, or contradictory hard rules	Check coverage requirements vs agent skills; the error text names the cause.
Leave balance drifting wrong	Balance updated outside `sync_leave_balance`, or missing balance row for the year	Confirm every path goes through the delta sync; verify a `leave_balances` row exists for that agent+year.
401 loops to login	Expired/invalid JWT; the response interceptor force-logs-out	Check token `exp`; re-login; verify `jwt_secret_key` matches between deploys.
Excel import rejected	A hand edit violates a hard constraint	Read the returned `violations` list — it names each broken rule.
Roster page blank on refresh	Missing SPA rewrite	Confirm `vercel.json` rewrites non-`/api` to `index.html`.
Connection pool exhausted	Sessions not closed / wrong Supabase pooler mode	Ensure `get_db` teardown runs; use Session pooler (5432).

Observability today: Uvicorn/Cloud Run request logs + pytest (16 suites) as the correctness net.
What's missing: structured logging, error tracking (Sentry), and distributed tracing — the honest "monitoring" answer and a §19 item.

19 · Future Improvements

Scale of change	What you'd do
10× larger	Add FK indexes + pagination; move solve to a background queue with a job-status endpoint; min-instances to remove cold starts; structured logging + Sentry; refresh-token rotation; rate limiting.
100× larger	Multi-tenancy (organization column + row-level isolation); Redis cache for public roster; read replicas; per-tenant solver workers; email/SMS notifications via a queue; feature flags.
Enterprise	SSO/OIDC + SCIM provisioning; audit export & compliance (SOC2); sharding by tenant; multi-region; solver as its own service with warm pools; real-time roster updates over WebSockets; fine-grained permissions beyond two roles.

Product-level: the `submitted_via = ai_parsed` enum and the "other" request type hint at a planned natural-language request intake (an agent types "I need next Friday off" and an LLM structures it) — a clean place to add a Claude-powered parser.

20 · Interview Preparation — 100+ Questions

Grouped by theme. Each: ideal answer · why it's asked · common wrong answer. Follow-ups are implied by the "why".

A. Architecture (1–12)

1. Describe the system in two minutes.

A modular-monolith FastAPI backend that turns weekly scheduling into a CP-SAT optimization problem, a Postgres store, and a React SPA; managers configure rules and generate/publish rosters, agents submit requests and view outcomes.

Tests whether you own the whole picture, not just pieces.

2. Why a monolith, not microservices?

One small team, one bounded domain; a monolith is simpler to deploy, test, and reason about. Clean internal layers mean I can split out the solver later if needed. Microservices would add network hops and ops burden for no benefit.

Checks maturity — resisting premature complexity.

3. Why no cache/queue/object-store?

Low read volume, synchronous sub-second solve, ephemeral files. Each would be complexity without benefit at this scale — and I can name the exact signal that would make me add each one.

Distinguishes cargo-culting from judgement.

Wrong answer: "I should have added Redis for best practice."

4. How does it scale horizontally?

The API is stateless (JWT, no session), so N instances behind a load balancer need no coordination; DB is the shared state.

5. Where's the single point of failure?

Postgres. Mitigate with replicas/failover; the app tier is replaceable.

6. Why FastAPI over Django?

Async, Pydantic validation, auto OpenAPI, and I don't need Django's ORM/admin/templates since the frontend is a separate SPA.

7. What's a modular monolith?

One deployable with strict internal module boundaries — the maintainability of services without the distributed-systems cost.

8. How are background jobs run without a queue?

Cloud Scheduler triggers Cloud Run Jobs that call idempotent lifecycle functions (auto-publish/lock).

9. Why keep the solver pure?

No DB/HTTP imports means it unit-tests in isolation and its logic is reasoned about without infrastructure.

10. What's the deployment topology?

Frontend on Vercel, backend container on Cloud Run, Postgres on Supabase, scheduled jobs on Cloud Scheduler.

11. How do frontend and backend communicate?

HTTPS JSON under `/api`; dev proxied by Vite, prod via `VITE_API_BASE_URL`; JWT in the Authorization header.

12. What would you extract into a service first?

The solver — it's CPU-bound and the natural async/worker boundary.

B. The Solver (13–30)

13. Why a constraint solver at all?

Scheduling is combinatorial with interacting rules; a solver does global search with backtracking and optimality, where greedy heuristics get stuck and are unfair.

The core differentiator of this project.

14. Hard vs soft constraints — give examples.

Hard: exactly-one-mode-per-day (no double-booking), weekly rest, shift headcount cap. Soft: honouring leave/off-day/shift-change/overtime, coverage, fairness.

15. How is no-double-booking encoded?

`AddExactlyOne` over the per-day mode variables {off, default, requested, poss…}.

16. Why is coverage soft, not hard?

A hard floor makes an understaffed week infeasible — the solver returns nothing. A shortfall variable absorbs missing agents and penalizes them heavily (weight 500, above all requests), so it degrades gracefully and only under-covers as a last resort.

The single best design decision to explain.

17. What is the objective function?

Minimize the weighted sum of "denied" indicators for each soft wish, plus a minimax fairness term, plus a scaled FCFS tie-breaker.

18. Why minimax fairness?

Minimizing total denials could pile them on one agent; minimizing the maximum per-agent denials spreads pain fairly.

19. How does FCFS not override real trade-offs?

Lexicographic scaling: multiply the main objective by (fcfs_cap+1) so any genuine 1-unit improvement outweighs the entire FCFS term; FCFS only settles exact ties.

Tests depth of optimization understanding.

20. Why scale weights by 100?

CP-SAT coefficients must be integers; weights carry 2 decimals, so ×100 preserves proportions.

21. How do overnight shifts work in coverage?

`shift_covers_slot` splits a wrapping shift into two same-day intervals and checks overlap.

22. What happens on a truly infeasible week?

Only hard constraints can cause it (e.g. required skill nobody holds); status=infeasible → 422 with a plain-English reason.

23. Why cap solve at 30s?

Bounds request latency; returns best-found feasible solution if optimality isn't proven in time.

24. How is leave different from an off-day?

Leave draws from a balance and stacks on top of the guaranteed rest day; an off-day just relocates the single rest day.

25. How is "honored" computed for multi-day leave?

All of its per-date condition variables must be true (`all( ... )`).

26. Why the "mode" variable design?

Compactness — one exactly-one choice per agent-day encodes double-booking prevention and shift selection with few variables.

27. Is CP-SAT deterministic?

Given the same input and seed, yes; and FCFS makes tie-breaking deterministic across otherwise-equal solutions.

28. Complexity of the model?

NP-hard generally; variable count O(agents×days×modes×skills); empirically fast at team scale, bounded by the cap.

29. How would you add "no more than 5 consecutive workdays"?

A sliding-window constraint over the off-variables — sum of "worked" in any 6-day window ≥ 1 off. Great follow-up to volunteer.

30. Why not machine learning?

This needs provable rule-satisfaction and optimality, not prediction; ML can't guarantee hard constraints.

C. Database (31–45)

31. Why relational?

Heavily related data + transactional multi-row writes + integrity constraints. Documents would push integrity into app code.

32. Explain a many-to-many here.

Agents↔Skills via the `agent_skills` join table.

33. What's normalized, what's denormalized?

Mostly 3NF; `satisfaction_metrics` stores a precomputed percentage snapshot — a justified mild denormalization.

34. Which unique constraints and why?

`(agent_id, year)` on balances, `week_start_date` on cycles, `email`, `skill.name` — prevent duplicates/ambiguity.

35. Where would you add indexes?

Hot FKs like `roster_assignments.roster_id`, `weekly_requests.week_cycle_id`.

Checks you understand query cost, not just schema.

36. How are transactions used?

`generate_roster` writes roster+assignments+balances+conflicts+metrics in one commit; a failure rolls all back.

37. What isolation level?

Postgres default read-committed; low write contention (single manager generating).

38. What's the N+1 problem and where did you avoid it?

Lazy relationships firing one query per parent; avoided with `joinedload` on agent skills/possible-shifts.

39. Why Alembic?

Reproducible, reversible, reviewable schema evolution vs unsafe auto-create.

40. Connection pooling — how?

One engine pools connections; each request borrows via `get_db` and returns on teardown; Supabase Session pooler in prod.

41. How would you handle two managers generating at once?

Today low-risk; add optimistic locking / a per-cycle advisory lock to serialize.

42. Why store `hashed_password` not password?

A DB leak must not expose credentials; bcrypt is one-way + salted.

43. Why enums in the DB?

Enforce a closed set of valid values at the storage layer, not just in code.

44. How do you migrate safely in prod?

Run Alembic `upgrade` on deploy; additive changes first, backfill, then constrain.

45. Sharding strategy?

By organization/tenant once multi-tenant.

D. Auth & Security (46–62)

46. Walk me through login.

Verify bcrypt hash → mint HS256 JWT with sub/role/agent_id/exp → client stores it → sent as Bearer on each request.

47. Is a JWT encrypted?

No — signed. Readable by anyone; never put secrets in it.

Extremely common misconception.

48. Can a user forge a manager token?

No — without the secret the HMAC signature won't verify.

49. Where's the token stored and what's the risk?

localStorage — convenient but XSS-reachable; mitigate with httpOnly cookies + CSRF or short tokens + refresh.

50. How do you revoke a token?

Today you can't before expiry — a known gap; fix with short TTL + refresh rotation or a denylist.

Tests honesty about JWT trade-offs.

51. Why is CSRF low-risk here?

Auth is a bearer header, not an auto-sent cookie, so cross-site requests can't attach it.

52. How is SQL injection prevented?

SQLAlchemy parameterization — no string-built SQL.

53. How is XSS prevented?

React auto-escapes; no `dangerouslySetInnerHTML`.

54. Client checks the role — is that security?

No, it's UX. Real enforcement is server-side `require_manager`.

55. How do you stop one agent reading another's data?

`/mine` endpoints filter by the token's `agent_id`, not a client-supplied id.

56. Where do secrets live?

Env vars / Secret Manager; never in the repo.

57. Brute-force protection?

Not yet — add rate limiting/lockout on login.

58. Why bcrypt not SHA-256?

bcrypt is deliberately slow + salted; fast hashes are trivially brute-forced.

59. File-upload security?

Parsed in memory + every row re-validated against hard constraints before commit.

60. Why an explicit CORS allow-list?

Wildcard + credentials is disallowed and unsafe; explicit origins only.

61. Audit trail — what and why?

Append-only log of actor/action/target/old-new/reason for accountability & debugging.

62. HTTPS — who terminates TLS?

Cloud Run and Vercel edge; tokens never travel in clear.

E. Frontend (63–76)

63. Redux or not — why?

No Redux; most state is server data, so React Query (a cache) handles it; a tiny Context holds auth.

64. Server state vs client state?

Server state is a cache of backend-owned data (React Query); client state is UI-only (useState/Context).

65. How does the SPA survive a page refresh on a deep link?

Vercel rewrite sends non-`/api` paths to `index.html`; the router re-resolves client-side.

66. How is auth persisted across reloads?

Token/role/agentId in localStorage, re-hydrated into Context on load.

67. How do protected routes work?

`ProtectedRoute` redirects unauthenticated/mismatched-role users; server still enforces.

68. How are API errors shown consistently?

`extractErrorMessage` normalizes FastAPI's three error shapes into one string.

69. Why react-hook-form?

Uncontrolled inputs → fewer re-renders; simple per-field validation.

70. What makes it a PWA?

vite-plugin-pwa generates a service worker + manifest → installable, offline shell.

71. Where's memoization used?

`useMemo` stabilizes the AuthContext value to avoid needless consumer re-renders.

72. How would you code-split?

Route-level `React.lazy` + Suspense; Vite already chunks vendors.

73. How is the 401 handled globally?

axios response interceptor clears storage and redirects to /login.

74. Why TypeScript?

Compile-time contract with the backend schemas; catches shape mismatches early.

75. How does the grid render?

A pure `lib` transform reshapes flat assignments into an agent×day grid; the component just maps it.

76. Why axios not fetch?

Interceptors (auth + 401), automatic JSON, and cleaner error objects.

F. Backend/Design patterns (77–88)

77. Name the layers.

Controller(routes) → Service → Repository(crud) → Domain/Solver → Model, with Schemas at the edge.

78. What is Dependency Injection here?

FastAPI `Depends` supplies db session, current user, role guards — testable and reusable.

79. Why separate schemas from models?

Never leak DB internals over the wire; validate input independently of persistence.

80. How are domain errors handled?

Typed exceptions with status_code+detail; routes map to HTTPException, some with a violations list.

81. Repository pattern benefit?

All queries per resource live in one place — testable, swappable, no SQL in routes.

82. How do you test the solver?

Feed dataclasses directly to `solve` — no DB — and assert assignments/decisions.

83. Why is `get_db` a generator?

Setup/teardown: yield a session, guarantee close after response.

84. Idempotency example?

Delta leave-sync + auto-publish/lock jobs are safe to re-run.

85. Singleton pattern?

`solver_weights` is a single id=1 config row via get-or-create.

86. Why is `domain/` shared by solver and import?

So both enforce identical coverage/leave rules — no drift.

87. State machine in the code?

Roster/cycle: draft→published→locked with guarded transitions.

88. How is config managed?

Pydantic Settings from env; fails fast on missing required vars.

G. Performance/Scale/Behavioral (89–105)

89. First bottleneck at 10k users?

DB connections + the synchronous solve blocking a worker; fix with a pooler and a job queue.

90. Add caching where?

Public roster reads (Redis), keyed by week; invalidate on publish.

91. Pagination — where and how?

List endpoints (requests, audit) via limit/offset or keyset.

92. How to remove cold starts?

Cloud Run min-instances ≥ 1.

93. Compression?

gzip/brotli at the edge for JSON/assets.

94. How would you make the solve async?

Enqueue a job, return 202 + job id, poll a status endpoint or push via WebSocket.

95. Multi-tenancy plan?

Add an organization column, scope every query, isolate per tenant, shard later.

96. Observability gaps?

Structured logs, Sentry, tracing — currently only request logs + tests.

97. Biggest technical risk?

Solver correctness/feasibility edge cases — mitigated by the pure model + unit tests.

98. What are you proudest of?

Modelling fairness (minimax) and FCFS (lexicographic) so neither corrupts the other, and making coverage degrade gracefully.

99. What would you do differently?

Single-transaction generate+audit; indexes/pagination from day one; refresh tokens; move token out of localStorage.

100. Hardest bug you'd anticipate?

Leave-balance double-charging on regeneration — solved by delta sync.

101. How did you validate correctness?

16 pytest suites incl. solver, lifecycle, excel round-trip, appeals.

102. Explain a trade-off you consciously accepted.

Stateless JWT gives free scaling at the cost of instant revocation — acceptable for 8h shifts.

103. Why soft coverage over hard — business framing?

"You don't have to strictly maintain minimum agents" — better a slightly-thin slot than no schedule at all.

104. How do you keep import and solver rules in sync?

Shared pure `domain/` functions used by both.

105. If the solve times out, what does the user get?

The best feasible solution found within 30s (still valid on all hard constraints), not an error.

21 · Deep “Why” Section

Component	Why created · Why here · Why this way · Why not otherwise · Why better/faster/secure
Pure solver `model.py`	Created to isolate the hardest logic; placed in its own module with no I/O so it's unit-testable; implemented with the compact mode design because fewer variables = faster solves; not embedded in the service because DB coupling would make it untestable; better because correctness is provable and tested, faster because the model is small, secure because it touches no external input directly.
Soft coverage + shortfall	Created so understaffing can't crash generation; here because coverage is the constraint most likely to be unsatisfiable; done with a penalized shortfall var; not a hard `≥` because that yields infeasible/no-answer; better because it degrades gracefully and still meets coverage whenever possible.
Lexicographic FCFS	Created to make tie-breaking fair & deterministic; implemented via magnitude scaling; not random because arbitrary tie-breaks feel unjust; better because it provably never overrides a real objective difference.
Delta leave-sync	Created so regeneration is idempotent; not absolute deduction because that double-charges; better because balances stay correct across any number of runs.
Stateless JWT	Created for scale-free auth; not server sessions because those need a shared store; secure via HMAC signature; trade-off: revocation.
React Query	Created to avoid hand-writing a cache; not Redux because server data ≠ client state; faster to build, less code, correct invalidation.
Shared `domain/`	Created so solver and Excel import can't disagree on the rules; better because there's one source of truth for coverage/leave semantics.
Alembic migrations	Created for safe schema evolution; not auto-create because it can't alter safely; better because changes are reviewable/reversible.

22 · Teach Me Like a Mentor — Master Recap

The five sentences that carry the whole interview

1. **What it is:** "An automated call-center rostering platform that models weekly scheduling as a CP-SAT optimization — hard rules as constraints, soft requests as weighted penalties — behind a FastAPI + Postgres backend and a React SPA."
2. **The clever core:** "Coverage is a *soft* constraint with a shortfall variable, so understaffing degrades gracefully instead of making the model infeasible."
3. **Fairness:** "Fairness is minimax over per-agent denials; first-come-first-served is a lexicographically-scaled tie-breaker that can never override a real trade-off."
4. **Correctness:** "The solver and domain rules are pure functions with no I/O, unit-tested, and shared by both roster generation and Excel re-validation so the two can't diverge."
5. **Judgement:** "I implemented the layers the problem needs and consciously omitted cache/queue/object-storage — and I can name the exact signal that would make me add each."

The one diagram to draw from memory

User → React SPA (Vercel) → FastAPI (Cloud Run): [CORS → route → `Depends` auth/role → endpoint → service → solver/CRUD → Postgres (Supabase)] → Pydantic JSON → React Query cache → UI. Off to the side: Cloud Scheduler → auto-publish/lock jobs. Draw that, label the solver box "hard=constraints, soft=weighted objective", and you've shown you understand the system.

Common confusions to never fall into

- JWTs are *signed, not encrypted*.
- The frontend role check is UX, not security.
- "No queue" doesn't mean "no background work".
- React Query is a cache, not a general state manager.
- The solver is exact optimization, not machine learning.
- Coverage is a soft target, not a hard floor.

Quick recap of the whole system

Managers configure **skills, shifts, agents, coverage, balances, and solver weights**, open a **weekly cycle**, and let agents submit **requests**. On **generate**, the service loads state, ranks requests FCFS, builds a **CP-SAT model** (hard: one-mode-per-day, weekly rest, headcount caps; soft: requests, coverage-shortfall, fairness, FCFS), solves within 30s, and writes a **roster** with assignments, flips request statuses, delta-syncs **leave balances**, and records **conflicts** and **satisfaction metrics** — all in one transaction. Rosters flow **draft** → **published** → **locked**, automatically on a Friday-night/Monday-morning schedule or manually. Agents view outcomes and **appeal**; every change is written to an **audit log**. The public sees the published roster with no login. It's deployed for ~$0 on Cloud Run + Supabase + Vercel.

Read this handbook twice, draw the diagram once without looking, and explain the five sentences aloud — you'll be ready to defend every decision.

Handbook generated from the CallRoster Pro repository. Every code snippet and file reference is drawn from the actual source; where behaviour could not be inferred from the code it was explicitly labelled as an assumption or a known gap. Solver behaviour reflects `backend/app/solver/model.py` and `service.py` as committed.